

Министерство науки и высшего образования Российской Федерации
Сибирский государственный университет науки и технологий
имени академика М. Ф. Решетнева

**В. В. Буряченко, А. Г. Зотин
А. И. Пахирка**

РАЗРАБОТКА ТРЕБОВАНИЙ ПРИ ПРОЕКТИРОВАНИИ ПРОГРАММНЫХ СИСТЕМ

*Утверждено редакционно-издательским советом
университета в качестве учебного пособия
для студентов бакалавриата по направлениям подготовки
09.03.04 «Программная инженерия»,
09.03.02 «Информационные системы и технологии»
всех форм обучения*

Красноярск 2019

УДК 004.4'2 004.4(075.8)
ББК 32.9я73
Б91

Рецензенты:

кандидат технических наук, доцент Е. Е. НОСКОВА
(Сибирский федеральный университет);
доктор технических наук, профессор М. Н. ФАВОРСКАЯ
(Сибирский государственный университет науки и технологий
имени академика М. Ф. Решетнева)

Буряченко, В. В.

Б91 Разработка требований при проектировании программных систем :
учеб. пособие / В. В. Буряченко, А. Г. Зотин, А. И. Пахирка ; СибГУ
им. М. Ф. Решетнева. – Красноярск, 2019. – 88 с.

Рассмотрены вопросы разработки и анализа требований при проектировании программных систем. Приведены основные понятия и классификация требований, представлены способы описания функциональных требований с использованием прецедентов, описаны основные модели языка UML, используемые в процессе разработки программного обеспечения.

Учебное пособие может использоваться как целостная составляющая или как часть практической подготовки по курсам «Разработка и анализ требований», «Проектирование и архитектура программных систем», «Базы данных» направлений подготовки 09.03.02 «Информационные системы и технологии», 09.03.04 «Программная инженерия» всех форм обучения, а также может быть полезно для студентов других направлений, изучающих основные принципы разработки и описания требований, представление требований к программному обеспечению в виде моделей UML.

УДК 004.4'2 004.4 (075.8)
ББК 32.9я73

© СибГУ им. М. Ф. Решетнева, 2019
© Буряченко В. В., Зотин А. Г., Пахирка А. И., 2019

ОГЛАВЛЕНИЕ

Предисловие	4
1. Информационные системы. Понятия и классификация требований	5
1.1. Понятие информационной системы	5
1.2. Роль требований в задаче внедрения автоматизированных информационных систем	11
Контрольные вопросы и задания	18
2. Рабочий поток определения требований	19
2.1. Модель требований	19
2.2. Свойства требований	24
Контрольные вопросы и задания	33
3. Моделирование прецедентов	34
3.1. Выявление актеров и прецедентов	34
3.2. Описание прецедентов	38
Контрольные вопросы и задания	49
4. Специфицирование требований	51
4.1. Структуризация прецедентов. Отношения между актерами и прецедентами	51
4.2. Диаграммы деятельности	57
Контрольные вопросы и задания	68
5. Методологии проектирования программного обеспечения	70
5.1. Планирование проекта на основе требований. Методология RUP	70
5.2. Принципы быстрой разработки программного обеспечения	72
Контрольные вопросы и задания	82
Послесловие	84
Библиографические ссылки	85
Библиографический список	85
Приложение	86

ПРЕДИСЛОВИЕ

В учебном пособии рассмотрены основные вопросы, касающиеся сбора и анализа требований. Пособие рассчитано на студентов бакалавриата, обучающихся по направлениям подготовки 09.03.04 «Программная инженерия» и 09.03.02 «Информационные системы и технологии», всех форм обучения. Материал, представленный в издании, содержит основные понятия и определения, различные модели в соответствии с синтаксисом языка Unified Modeling Language (UML) и дополнен примерами различных задач разработки программного обеспечения.

Учебное пособие состоит из предисловия, пяти глав, послесловия и списка рекомендуемой литературы, который включает как широко известные в данной области труды, так и современные учебно-методические издания.

В первой главе рассмотрены основные понятия информационных систем и требования к программному обеспечению. Приводится классификация требований согласно К. Вигерсу. Рассмотрено разделение требований по уровням, а также на функциональные и нефункциональные.

Во второй главе описаны различные этапы работы с требованиями, а также документы, которые используются на разных стадиях разработки программных систем. Кроме того, приведены основные свойства и атрибуты требований.

Третья глава посвящена одному из наиболее эффективных способов описания и документирования требований: представление в виде прецедентов. Здесь даны понятия актера и прецедента (варианта использования), глоссария системы, а также приводится один из наиболее известных шаблонов описания прецедента.

Четвертая глава содержит описание различных моделей языка UML, которые используются для специфицирования требований, например, такие модели, как диаграмма прецедентов, диаграмма активности (деятельности), диаграмма состояний. Приводятся примеры из области описания практической задачи при реализации приложения «Интернет-магазин». В заключительной главе учебного пособия описаны известные методологии разработки программного обеспечения, направленные на использование большой группой разработчиков, а также методологии быстрой (гибкой) разработки.

Теоретический материал в каждой главе дополняется контрольными вопросами, направленными на проверку усвоения материала читателями.

1. ИНФОРМАЦИОННЫЕ СИСТЕМЫ. ПОНЯТИЯ И КЛАССИФИКАЦИЯ ТРЕБОВАНИЙ

1.1. ПОНЯТИЕ ИНФОРМАЦИОННОЙ СИСТЕМЫ

Информационной системой (ИС) либо автоматизированной ИС (АИС) будем называть программно-аппаратную систему, предназначенную для автоматизации целенаправленной деятельности конечных пользователей, обеспечивающую в соответствии с заложенной в неё логикой обработки возможность получения, модификации и хранения информации.

Ключевым моментом в этом определении является понятие целенаправленной деятельности. Речь идёт о деятельности, направленной на решение конкретной задачи, стоящей перед пользователем (коллективом пользователей).

Некоторые исследователи определяют ИС несколько иным образом. ИС в широком смысле – взаимосвязанная совокупность средств, методов и персонала, используемых для хранения, обработки и выдачи информации в интересах достижения поставленной цели.

Информационные системы могут быть классифицированы по различным признакам. Рассмотрим наиболее важные из них.

Классификация по масштабу. По масштабу ИС подразделяются однопользовательские, групповые и корпоративные.

Однопользовательские ИС предназначены для использования на одном рабочем месте. В настоящее время на мировом и отечественном рынке представлено множество решений, предназначенных для автоматизации деятельности отдельно взятого пользователя. Как правило, это решения, ориентированные на специалиста в той или иной области, будь то составление спецификаций для сборки изделий из комплектующих, планирование ремонтов оборудования, учёт расходов и доходов частного предпринимателя оптовой торговли, либо составление расписаний занятий в деканате.

В настоящее время альтернативу таким узкоспециализированным системам составили табличные процессоры, не имеющие проблемной специализации, в первую очередь, MS Excel. Системы этого класса трудно отнести к классу ИС, но зачастую они позволяют непрограммирующему специалисту создавать и, что очень важно, самостоятельно развивать собственные решения, заменяющие, а местами и перекрывающие, функционал однопользовательских систем образца 1990-х годов.

В основе большинства однопользовательских систем лежит стандарт X-Base (Clipper, FoxPro, dBase). Широко используются также решения на базе систем Paradox, Clarion, MS Access. Каждая из перечисленных конкурирующих систем обладает собственной высокоуровневой инструментальной средой, позволяющей спроектировать базу данных, логику обработки, пользовательский интерфейс, отчёты с помощью мастеров-построителей. На рубеже тысячелетий появились также и однопользовательские решения на базе промышленных реляционных систем управления базами данных (СУБД). В этом случае программное обеспечение (ПО) сервера устанавливается непосредственно на рабочую станцию пользователя. Примером может служить Personal Oracle и широко известная SQLite. Данные решения предъявляют значительные требования к ресурсам рабочей станции, однако несут в себе многие преимущества промышленных СУБД.

Групповые системы предназначены для автоматизации деятельности в рабочей группе (отделе, кластере, группе проекта и т. д.). В отличие от однопользовательских ИС, групповые системы, как правило, представляют специализированные клиентские решения (их часто называют автоматизированными рабочими местами (АРМ)) для различных участников группы. Например, для оптовой фирмы ИС может представлять набор таких АРМ, как «менеджер по продажам», «кладовщик», «снабженец», «директор». Для учебного планирования – «преподаватель», «работник бюро планирования», «работник учебного отдела», «специалист по планированию на кафедре», «работник деканата».

Групповое использование решений на базе табличных процессоров возможно, но имеет существенные ограничения, связанные с разграничением доступа, регламентацией и синхронизацией вносимых изменений. Фактически, единственный режим их использования, обеспечивающий корректность данных, – «файловый сервер, один автор, N читателей». При создании групповых ИС в целом используются те же средства и инструментальные среды, что и при создании однопользовательских ИС. Следует, однако, отметить, что для использования в группе при выборе между системами с файловым и реляционным сервером следует отдавать предпочтение реляционному серверу, причём целесообразно использование выделенного сервера. Это может быть, например, сервер Oracle, DB2, MS SQL, Sybase, Informix.

Корпоративные ИС (КИС) предназначены для автоматизации деятельности предприятия. В англоязычной литературе понятие КИС

неразрывно связано с понятием ERP (Enterprise Resource Planning). В основе ERP-систем лежит международный стандарт управления предприятием MRP-II (Manufacture Resource Planning), обеспечивающий возможность учета, анализа и планирования основных ресурсов – финансовых, человеческих, материальных. Соответственно, корпоративные ERP-системы – это набор интегрированных приложений, которые комплексно, в едином информационном пространстве поддерживают все основные аспекты управленческой деятельности предприятий: планирование ресурсов (финансовых, человеческих, материальных) для производства товаров (услуг), оперативное управление выполнением планов (включая снабжение, сбыт, ведение договоров), все виды учета и анализ результатов хозяйственной деятельности.

Среди требований, предъявляемых к современным КИС, можно выделить:

- централизацию данных в единой базе (в основе – всегда промышленная СУБД);
- близкий к реальному времени режим работы;
- сохранение общей модели управления для предприятий разных отраслей;
- поддержка территориально распределенных структур;
- работа на широком круге аппаратно-программных платформ и СУБД.

Классификация по архитектуре. В зависимости от типа архитектуры выделяют следующие ИС:

1. Архитектура «файл–сервер». Это исторически первая архитектура информационных систем. Как исполняемые модули, так и данные размещаются в отдельных файлах операционной системы. Доступ

к данным осуществляется путём указания пути (path) и использования файловых операций (открыть, считать, записать). Для хранения данных используется выделенный сервер (отдельный компьютер), который и является файловым сервером. Исполняемые модули хранятся либо на рабочих станциях, либо на файловом сервере. В последнем случае упрощается процедура их администрирования, но при этом возрастают требования к надёжности сети.

2. Архитектура «клиент–сервер». «Клиент–сервер» – это не только архитектура, это новая парадигма, пришедшая на смену устаревшим концепциям. Суть её заключается в том, что клиент (исполняемый модуль) запрашивает те или иные сервисы в соответствии

с определённым протоколом обмена данными. При этом, в отличие от ситуации с файловым сервером, нет необходимости в использовании прямых путей операционной системы: клиент их «не знает», ему «известны» лишь имя источника данных и другие специальные сведения, используемые для авторизации клиента на сервере. Сервер, который физически может находиться на том же компьютере, или на другом конце земного шара, обрабатывает запрос клиента и, производя соответствующие манипуляции с данными, передаёт клиенту запрашиваемую порцию данных.

В рамках направления «клиент–сервер» существуют два основных «диалекта» – тонкий и толстый клиент.

В системах на основе тонкого клиента используется мощный сервер баз данных, это – высокопроизводительный компьютер и библиотека так называемых хранимых процедур, позволяющих производить вычисления, реализующие основную логику обработки данных непосредственно на сервере. Клиентское приложение, соответственно, предъявляет невысокие требования к аппаратному обеспечению рабочей станции. Основное достоинство таких систем – относительная дешевизна клиентских станций.

Системы с толстым клиентом, напротив, реализуют основную логику обработки на клиенте, а сервер представляет собой в чистом виде сервер баз данных, обеспечивающий исполнение только стандартизованных запросов на манипуляцию с данными (как правило, чтение, запись, модификацию данных в таблицах реляционной базы данных). В системах такого класса требования к рабочей станции выше, а к серверу – ниже. Достоинство архитектуры заключается в переносимости серверного компонента на серверы различных производителей: все промышленные серверы баз данных реляционного типа поддерживают работу со стандартизованным языком манипулирования данными SQL, но внутренний встроенный язык обработки данных, необходимый для реализации логики обработки на сервере, у каждого из серверов свой.

3. Трёхслойная архитектура. Базируется на дальнейшей специализации компонентов архитектуры: клиент занимается только организацией интерфейса с пользователем, сервер баз данных – только стандартизованной обработкой данных. Для реализации логики обработки данных архитектура предусматривает отдельный слой – слой бизнес-логики. Этот слой может представлять собой либо выделенный сервер (сервер приложений), либо размещаться на клиенте

в качестве динамической библиотеки. Данная архитектура позволила соединить достоинства тонкого и толстого клиентов: хорошая переносимость соединяется в ней с невысокими требованиями к клиенту.

С развитием интернет-технологий появилась разновидность трёхслойной архитектуры на основании использования web-технологий. В этой разновидности роль сервера приложений играет web-сервер, а в качестве клиента используется стандартный web-браузер. Достоинства этой архитектуры – в пониженных требованиях к клиенту и в легкой встраиваемости данной архитектуры в мировые информационные сети. Основной недостаток – это известные ограничения, накладываемые на интерфейс пользователя web-браузерами.

Классификация по характеру использования информации. По характеру использования информации с некоторой степенью приближения все ИС можно разделить на два класса: информационно-поисковые и управляющие.

Конечные пользователи *информационно-поисковых систем* (ИПС), как правило, имеют доступ к хранимым данным только «по чтению» и используют данные системы для поиска ответов на те или иные вопросы. Доступ по модификации данных имеет администратор системы, в функции которого входит обеспечение актуальности информации, устранение ошибок.

Классические примеры ИПС – системы поиска в библиотеках, на транспорте (справки о наличии билетов). На современном этапе развития информационных технологий классические ИПС постепенно вытесняются поисковыми серверами Интернета общего назначения и специализированными.

Альтернатива ИПС – *управляющие системы* – автоматизируют (полностью или частично) деятельность, связанную с принятием решений. Действия конечных пользователей таких систем приводят к модификации информации, что, конечно, не исключает возможности просто получать информацию, как в ИПС.

Примеры управляющих систем – системы бухгалтерского учета, системы планирования производственных ресурсов и т. п.

Классификация по поддерживаемым стандартам управления и технологиям коммуникации. Эпоха стихийной разработки АИС закончена. Современные автоматизированные информационные системы разрабатываются, исходя из сложившихся реалий автоматизированного управления бизнесом. Существует значительное количество концепций, технологий, подходов, нашедших своё эффективное применение в различных отраслях промышленности по всему миру.

Некоторые из них приобрели статус международных стандартов. В спецификации АИС, разрабатываемой для массовой продажи, как правило, указывается, какие стандарты и технологии управления она поддерживает. Менее строгие требования предъявляются к АИС, создаваемым под заказ для конкретного предприятия. Однако и в этом случае не учитывать сложившийся в мире позитивный опыт просто неразумно. Ниже перечислены некоторые наиболее важные технологии и стандарты:

- MRP (Material Requirements Planning) – планирование поставок материалов, исходя из данных о комплектации производимой продукции и плана продаж;

- CRP (Capacity Requirements Planning) – планирование производственных мощностей, исходя из данных о технологии производимой продукции и прогноза спроса;

- MRP-II (Manufacture Resource Planning) – планирование материальных, мощностных и финансовых ресурсов, необходимых для производства; стандартизовано ISO;

- ERP (Enterprise Resource Planning) – финансово-ориентированное планирование ресурсов предприятия, необходимых для получения, изготовления, отгрузки и учёта заказов потребителей на основе интеграции всех отделов и подразделений компании;

- SCM (Supply Chain Management) – управление цепочками поставок; реализация бизнес-процессов на базе внешних предприятий и торговых площадок; основано на референтной модели SCOR, стандартизированной Supply Chain Council;

- CRM (Customer Relationship Management) – управление взаимоотношениями с заказчиками; комплекс методов и средств, нацеленных на завоевание, удовлетворение требований и сохранение платежеспособных клиентов;

- ERP-II (Enterprise Resource & Relationship Processing) – управление ресурсами и взаимоотношениями предприятия; объединяет в себе три вышеперечисленные технологии;

- Workflow – технология, управляющая потоком работ при помощи программного обеспечения, способного интерпретировать описание процесса, взаимодействовать с его участниками и при необходимости вызывать соответствующие программные приложения;

- OLAP (Online Analytical Processing) – оперативный анализ данных; технология поддержки принятия управленческих решений на основе концепции многомерных кубов информации;

– Project Management – управление проектами; поддерживается рядом международных стандартов;

– CALS (Continuous Acquisition and Lifecycle Support) – непрерывная информационная поддержка поставок и жизненного цикла; описывает совокупность принципов и технологий информационной поддержки жизненного цикла продукции на всех его стадиях, объединяет в себе практически все вышеперечисленные подходы и технологии.

1.2. РОЛЬ ТРЕБОВАНИЙ В ЗАДАЧЕ ВНЕДРЕНИЯ АВТОМАТИЗИРОВАННЫХ ИНФОРМАЦИОННЫХ СИСТЕМ

Не говоря о многообразии производителей АИС, на основании только сформулированных признаков становится очевидным, что существует значительное количество АИС, и данные АИС существенно различаются между собой. Следовательно, выбор АИС для предприятия – достаточно нетривиальная задача. Для того чтобы успешно её решить, необходимо хорошо знать объект внедрения (автоматизируемое предприятие), особенности его деятельности, стратегию развития и многие другие аспекты, предопределяющие характеристики закупаемой АИС. Указанные знания, в конечном счете, формализуются в документе требований к АИС, на основе которого и осуществляется выбор и последующая настройка АИС. В ещё большей степени требования к АИС важны при разработке ее на заказ.

Л. Новиков в русской редакции нотации RUP приводит следующее *определение требования*: «Требование – это условие или возможность, которой должна соответствовать система». В IEEE Standard Glossary of Software Engineering Terminology (1990) данное понятие трактуется шире. Требование – это:

1) условия или возможности, необходимые пользователю для решения проблем или достижения целей;

2) условия или возможности, которыми должна обладать система или системные компоненты, чтобы выполнить контракт или удовлетворять стандартам, спецификациям или другим формальным документам;

3) документированное представление условий или возможностей для пп. 1 и 2.

Введём ещё одно определение. *Требования* – это исходные данные, на основании которых проектируются и создаются автоматизированные информационные системы. Первичные данные поступают

из различных источников, характеризуются противоречивостью, неполнотой, нечёткостью, изменчивостью. Требования нужны, в частности, для того, чтобы разработчик мог определить и согласовать с заказчиком временные и финансовые перспективы проекта автоматизации, поэтому значительная часть требований должна быть собрана и обработана на ранних этапах создания АИС. Однако собрать на ранних стадиях все данные, необходимые для реализации АИС, удаётся только в исключительных случаях. На практике процесс сбора, анализа и обработки растянут во времени на протяжении всего жизненного цикла АИС, зачастую нетривиален и содержит множество подводных камней.

Классификация требований. Существует значительное количество различных методов классификации требований, наиболее существенные из которых – это требования к продукту и процессу, а также требования к проекту. В своей основе требования – это то, что формулирует заказчик. Цель, которую он преследует, заключается в том, чтобы получить хороший конечный продукт – функциональный и удобный в использовании. Поэтому *требования к продукту* являются основополагающим классом требований. Более подробно требования к продукту детализируются в описанных ниже классификациях.

Вопросы формулирования *требований к проекту*, т. е. к тому, как разработчик будет выполнять работы по созданию целевой системы, казалось бы, не лежат в компетенции заказчика. Без регламентации процесса заказчиком легко можно было бы обойтись, если бы все проекты всегда выполнялись точно и в срок. Однако, к сожалению, мировая статистика результатов программных проектов говорит об обратном. Заказчик, вступая в договорные отношения с разработчиком, несёт различные риски, главными из которых является риск получить продукт с опозданием либо ненадлежащего качества. Основные мероприятия по контролю и снижению риска – регламентация процесса создания программного обеспечения и его аудит. Насколько подробно заказчику следует регламентировать требования к проекту – вопрос риторический. Ответ на него зависит от множества факторов, таких как ценность конечного продукта для заказчика, степень доверия заказчика к разработчику, сумма подписанного контракта, увязка срока сдачи продукта в эксплуатацию с бизнес-планами заказчика и т. д. Однако со всей определённой можно сказать следующее: регламентация процесса заказчиком позволяет снизить его риски, мероприятия заказчика по регламентации процесса приводят к дополнительным

накладным расходам. Требуется найти разумный компромисс между степенью контроля рисков и величиной расходов.

В качестве требований к проекту может быть внесён регламент отчётов разработчика, совместных семинаров по оценке промежуточных результатов, определены характеристики компетенций участников рабочей группы, исполняющих проект, их количество, указана методология управления проектом. Приведём пример формулировки требования к оффшорному проекту (заказчик и разработчик физически находятся в разных государствах), в этой ситуации заказчику требуется жёсткий контроль над разработчиком:

1. Разработчик представляет заказчику согласованный план работ с детализацией (Work Breakdown Structure – WBS) с точностью до конкретных исполнителей.

2. Разработчик осуществляет ежедневные сборки, регрессионное тестирование компонентов разрабатываемого продукта и тестирование продукта в целом.

3. Все управленческие и проектные артефакты, исходные коды и тестовые примеры размещаются в режиме online в интегрированной среде разработки Rational ClearCase® с возможностью для заказчика осуществления online-мониторинга на базе web-технологий.

Уровни требований. Внедрение ИС на предприятии всегда преследует конкретные бизнес-цели, такие как, например, повышение прозрачности бизнеса, сокращение сроков обработки информации, экономия накладных расходов и т. д. Современные информационные системы – это крупные программные системы, содержащие в себе множество модулей, функциональных, интерфейсных элементов, отчётов и т. д. Как охватить такие разнородные вещи, как цели, преследуемые топ-менеджером предприятия заказчика, описание интерфейса пользователя и характеристики модуля, осуществляющего расчёт себестоимости изделия?

К счастью, человечество уже давно изобрело приёмы борьбы со сложностями, широко применяемые в моделировании сложных объектов, такие как абстракция и декомпозиция. Применительно к дисциплине анализа требований к программным системам эти принципы работают следующим образом. Требования разделяются по уровням. Уровни требований связаны, с одной стороны, с уровнем абстракции системы, с другой – с уровнем управления на предприятии.

Обычно выделяют три уровня требований. На верхнем уровне представлены так называемые *бизнес-требования* (business requirements). Например, бизнес-требование – система должна сократить срок

оборачиваемости обрабатываемых на предприятии заказов в три раза. Бизнес-требования обычно формулируются топ-менеджерами либо акционерами предприятия.

Следующий уровень – *уровень требований пользователей* (user requirements). Например, требование пользователя – система должна предоставлять диалоговые средства для ввода исчерпывающей информации о заказе, последующей фиксации информации в базе данных и маршрутизации информации о заказе сотруднику, отвечающему за его планирование и исполнение. Требования пользователей часто бывают плохо структурированными, дублирующимися, противоречивыми. Поэтому для создания системы важен третий уровень, в котором осуществляется формализация требований.

Третий уровень – *функциональный* (functional requirements). Например, функциональные требования (или просто функции) по работе с электронным заказом – заказ может быть создан, отредактирован, удалён и перемещён с участка на участок.

Существуют объективные противоречия между требованиями различных уровней. Так, очевидным бизнес-требованием является требование о полноте информации, собираемой на рабочих местах пользователей в единую базу данных. Чем полнее информация, тем глубже база для анализа деятельности и принятия решений. С другой стороны, для конкретного пользователя системы вполне может быть достаточным использование только той части информации, которая влияет на выполнение его основных функций.

Важными правилами внедрения и использования АИС на предприятии являются следующие: одна точка сбора, данные собираются там, где они появляются. Использование этих правил позволяет избежать затрат на необоснованное дублирование информации и, что важнее, потерь от ошибок учёта, неизбежно возникающих при дублировании точек ввода. Внедрение АИС на предприятии приводит к необходимости оснащения всех точек ввода информации автоматизированными рабочими местами, обучению персонала и, зачастую, оптимизации и повышению уровня формализации рабочих процессов, выполняемых персоналом. Поэтому внедрение АИС – это непростой процесс, часто требующий «перекройки» человеческого сознания и встречающий сопротивление со стороны пользователей, которые не готовы либо не хотят работать по-новому.

Существуют различные трактовки понятия системных требований (system requirements). К. Вигерс формулирует данный термин как «высокоуровневые требования к продукту, которые содержат многие

подсистемы, т. е. системе» [1]. При этом под системой понимается программная, программно-аппаратная либо человеко-машинная система. Данная система является сложной, структурированной системой, и системные требования являются подмножеством функциональных требований к продукту. В это подмножество целесообразно относить наиболее важные, существенные требования, которые относятся в целом к системе и не содержат избыточной детализации.

В практике компьютерной инженерии бытует другой, более узкий контекст использования данного понятия: под *системными требованиями в узком смысле* понимаются требования, выдвигаемые прикладной программной системой (в частности, информационной) к среде своего функционирования (системной, аппаратной). Примером таких требований является тактовая частота процессора, объём памяти, требования к выбору операционной системы.

Функциональные, нефункциональные требования и характеристики продукта. *Функциональные требования* регламентируют функционирование или поведение системы (behavioral requirements). Функциональные требования отвечают на вопрос, что должна делать система в тех или иных ситуациях. Функциональные требования определяют основной «фронт работ» разработчика и устанавливают цели, задачи и сервисы, предоставляемые системой заказчику.

Функциональные требования записываются, как правило, посредством предписывающих правил, например, «система должна позволять кладовщику формировать приходные и расходные накладные». Другим способом являются так называемые *варианты использования* (uses cases) – популярный и весьма продуктивный способ представления требований.

Это основной, определяющий вид требований, который будет рассматриваться в данном учебном пособии.

Нефункциональные требования, соответственно, регламентируют внутренние и внешние условия или атрибуты функционирования системы. К. Вигерс выделяет следующие группы нефункциональных требований:

- внешние интерфейсы (External Interfaces);
- атрибуты качества (Quality Attributes);
- ограничения (Constraints).

Среди *внешних интерфейсов* в большинстве современных АИС наиболее важным является интерфейс пользователя (User Interface – UI). Кроме того, выделяются интерфейсы с внешними устройствами (аппаратные интерфейсы), программные интерфейсы и интерфейсы передачи информации (коммуникационные интерфейсы).

Основные *атрибуты качества* – применимость, надежность, производительность, эксплуатационная пригодность – достаточно хорошо раскрыты в модели FURPS.

Ограничения – это формулировки условий, модифицирующих требования или наборы требований, при сужении выбора возможных решений по их реализации, выбор платформы реализации и (или) развертывания (протоколы, серверы приложений, баз данных), которые, в свою очередь, могут относиться, например, к внешним интерфейсам.

К. Вигерс формулирует *характеристику* (feature) как набор логически связанных функциональных требований, которые обеспечивают возможности пользователя и удовлетворяют бизнес-цели.

С точки зрения инженерии требований feature является самостоятельным артефактом, который может быть соотнесен как с функциональными требованиями, так и с нефункциональными.

Роль характеристик проявляется в первую очередь в области маркетинга: не всякий потенциальный потребитель продукта станет читать его функциональные описания, а набор ключевых характеристик, характеризующих конкурентные преимущества, можно сделать лаконичным и уместить на одной странице рекламной листовки либо напечатать на компакт-диске.

Взаимосвязь различных уровней и видов требований согласно монографии К. Вигерса [1] представлена на рис. 1.1. Примерная структура требований представлена в приложении.

Классификация RUP. В спецификациях Rational Unified Process (RUP) при классификации требований используется модель FURPS+ со ссылкой на стандарт IEEE Std 610.12.1990.

Аббревиатура FURPS обозначает следующие категории требований:

- Functionality (функциональность);
- Usability (применимость);
- Reliability (надёжность);
- Performance (производительность);
- Supportability (эксплуатационная пригодность).

Символ «+» расширяет FURPS-модель, добавляя к ней:

- ограничения проекта;
- требования выполнения;
- требования к интерфейсу;
- физические требования.

Часть из них уже была рассмотрена выше.

Кроме того, в спецификациях RUP выделяются такие категории требований, как требования, указывающие на необходимость согласованности с некоторыми юридическими и нормативными актами, требования к лицензированию, требования к документированию.



Рис. 1.1. Уровни и виды требований

Методологии и стандарты, регламентирующие работу с требованиями. Среди основополагающих нормативных документов в области работы с требованиями можно выделить следующие:

1. Разработки IEEE:
 - IEEE 1362 «Concept of Operations Document»;
 - IEEE 1233 «Guide for Developing System Requirements Specifications»;
 - IEEE Standard 830–1998 «IEEE Recommended Practice for Software Requirements Specifications»;
 - IEEE Standard Glossary of Software Engineering Terminology / IEEE Std 610.12–1990;

– IEEE Guide to the Software Engineering Body of Knowledge – SWEBOOK®.

2. Отечественные ГОСТы:

– ГОСТ 34.601–90 «Информационная технология. Автоматизированные системы. Стадии создания»;

– ГОСТ 34.602–89 «Информационная технология. Техническое задание на создание автоматизированной системы»;

– ГОСТ 19.201–78 «Единая система программной документации. Техническое задание. Требования к содержанию и оформлению».

Контрольные вопросы и задания

1. Охарактеризуйте понятие информационной системы.
2. Дайте классификацию информационных систем по масштабу.
3. Приведите классификацию информационных систем по используемой архитектуре.
4. Какие стандарты и технологии управления и планирования производства вам известны?
5. Дайте определение понятия требований.
6. В чем различие между требованиями к продукту и проекту?
7. Охарактеризуйте уровни требований.
8. Что понимается под системными требованиями?
9. В чем различия между функциональными и нефункциональными требованиями?
10. Приведите понятие функциональных требований в узком смысле.
11. Опишите модель требований FURPS и FURPS+. В какой методологии проектирования она используется?

2. РАБОЧИЙ ПОТОК ОПРЕДЕЛЕНИЯ ТРЕБОВАНИЙ

2.1. МОДЕЛЬ ТРЕБОВАНИЙ

Как показано на рис. 2.1, большая часть работы в процессе определения требований выполняется в фазах начала и уточнения в самом начале жизненного цикла проекта. Это и неудивительно, поскольку невозможно продвигаться вперед за фазу уточнения, пока неизвестно, что предполагается разрабатывать.

До начала работы над объектно-ориентированным (ОО) анализом и проектированием уже необходимо иметь некоторое представление о возможностях системы, которая должна получиться в результате. Именно в этом состоит цель рабочего потока определения требований. С точки зрения аналитика или дизайнера цель ОО-анализа – это поиск и достижение соглашения о функциях системы, написанного на языке пользователей системы. Создание высокоуровневой спецификации того, что должна делать система, – это часть процесса выработки требований (requirements engineering).

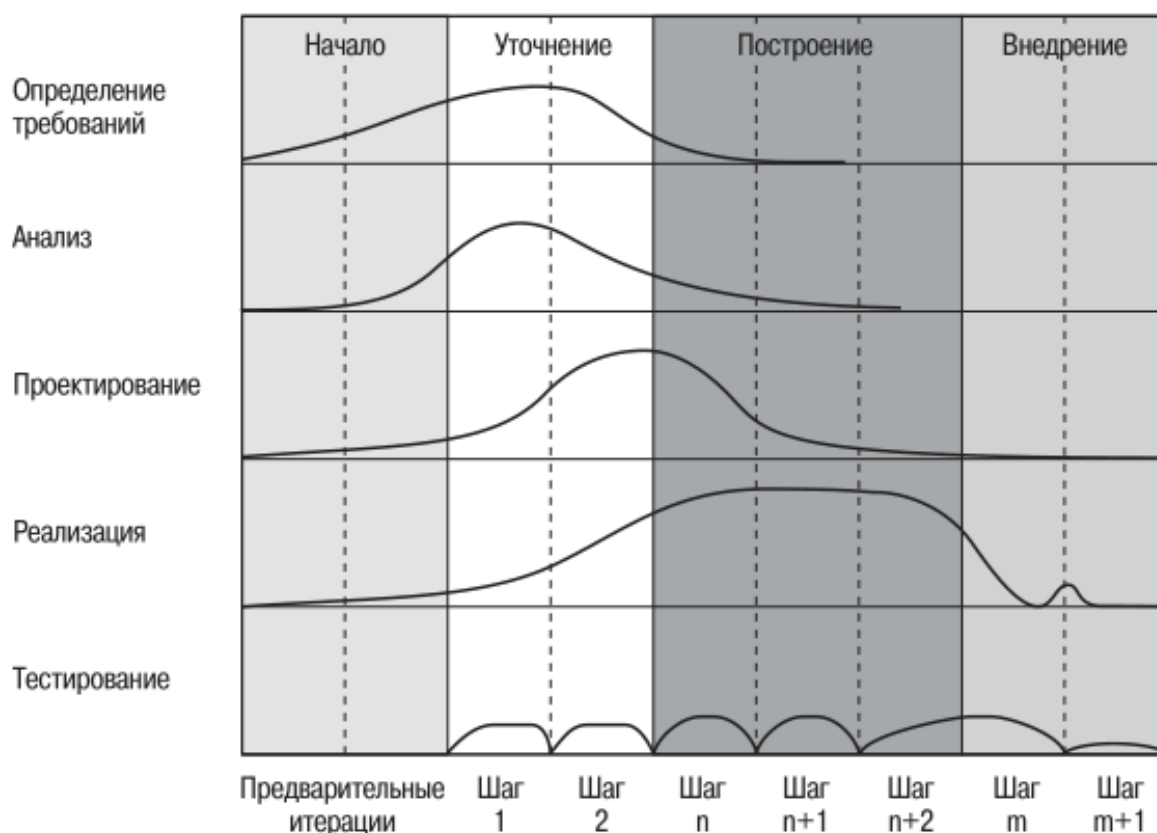


Рис. 2.1. Рабочие потоки программной инженерии

Большая часть работы с требованиями проводится в начале проекта, в фазах начала и уточнения.

У любой заданной системы может быть множество различных заинтересованных сторон: разные типы пользователей, специалисты по эксплуатации, штат по обслуживанию, продаже, управлению и т. д. Выработка требований состоит в выявлении и классификации требований, предъявляемых к системе заинтересованными сторонами. Это переговорный процесс, поскольку часто выдвигаются противоречивые требования, которые должны быть согласованы. Например, одной группе хочется добавлять большое количество пользователей, что может привести к нереальному трафику в существующей базе данных и инфраструктуре связи. В настоящее время это наиболее распространенный конфликт, поскольку все большее и большее число компаний открывают части разработанных ими систем громадной аудитории пользователей через Интернет.

Некоторые книги по UML отмечают, что прецеденты UML являются единственным способом фиксирования требований. Но это утверждение оказывается неверным при более тщательном рассмотрении. Прецеденты могут отражать только функциональные требования, которые являются описанием того, что будет делать система. Однако есть и другие требования, не относящиеся к функциональности, которые являются описаниями ограничений, накладываемых на систему (производительность, надежность и т. п.), и не могут быть представлены в виде прецедентов. Поэтому предлагается надежный подход к выработке требований, иллюстрирующий дополнительные эффективные способы выявления обоих видов требований.

Метамодель подхода к выработке требований показана на рис. 2.2. Здесь используется синтаксис UML, который будет рассмотрен позже. На диаграмме используются следующие обозначения:

1. Пиктограммы, напоминающие папки, – это пакеты UML. Они являются механизмом группировки UML и содержат группы элементов модели UML. В сущности, они очень напоминают настоящие папки файловой системы тем, что используются для организации и группировки взаимосвязанных сущностей. Маленький треугольник в верхнем правом углу пакета указывает на то, что в пакете находится модель.

2. Пиктограмма якоря показывает, что сущность, изображенная со стороны кружка, содержит сущность, находящуюся на другом конце линии.

Данная метамодель показывает, что *спецификация требований* к программному обеспечению (Software Requirements Specification – SRS) включает *модель требований* (Requirements Model) и *модель прецедентов* (Use Case Model). Эти две модели являются разными, но взаимодополняющими способами представления требований, предъявляемых к системе. Можно видеть, что в модель требований входят функциональные требования (требования, определяющие, что должна делать система) и нефункциональные требования (требования, выражающие ограничения системы, не относящиеся к ее функциональности).

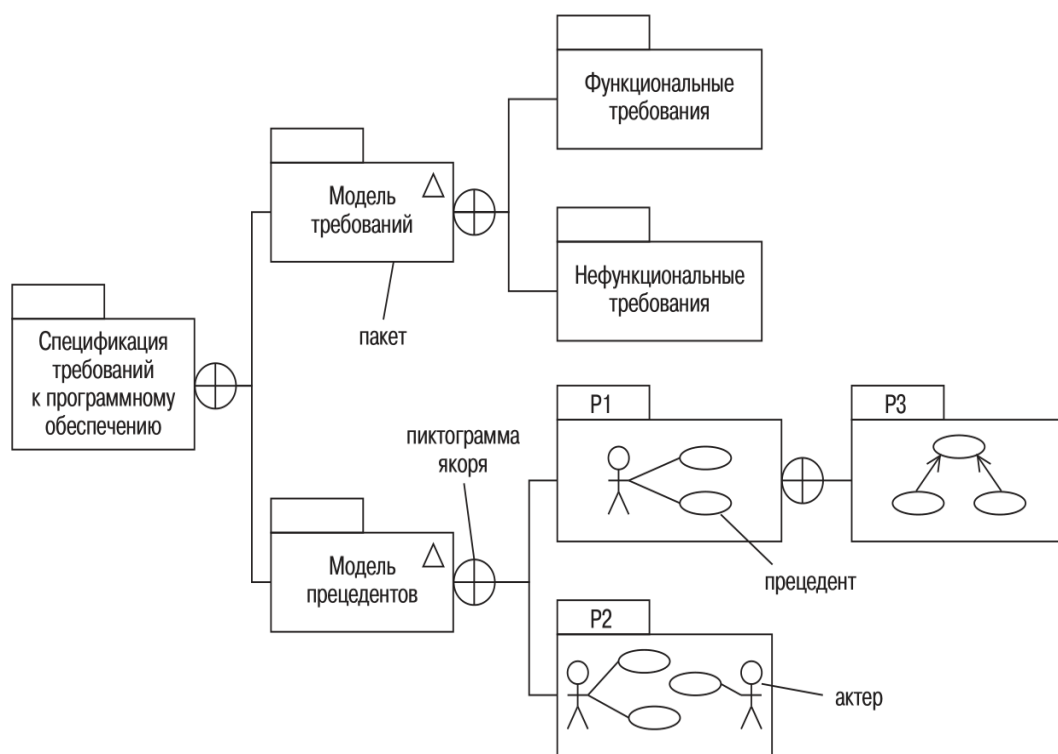


Рис. 2.2. Метамодель требований

Модель прецедентов включает множество пакетов прецедентов (на рис. 2.2 показаны только три из них), которые содержат прецеденты (спецификации функциональных возможностей системы), актеров (внешние роли, непосредственно взаимодействующие с системой) и отношения. По сути, SRS – это начальная стадия процесса построения программного обеспечения. Это отправная точка ОО-анализа и проектирования.

Конкретные задачи рабочего потока определения требований в UP показаны на рис. 2.3. Такие диаграммы называют детализацией рабочего потока, поскольку они детализируют составляющие

задачи определенного потока работ. Детализация рабочего потока показывает исполнителей и деятельность, вовлеченных в конкретный рабочий поток.

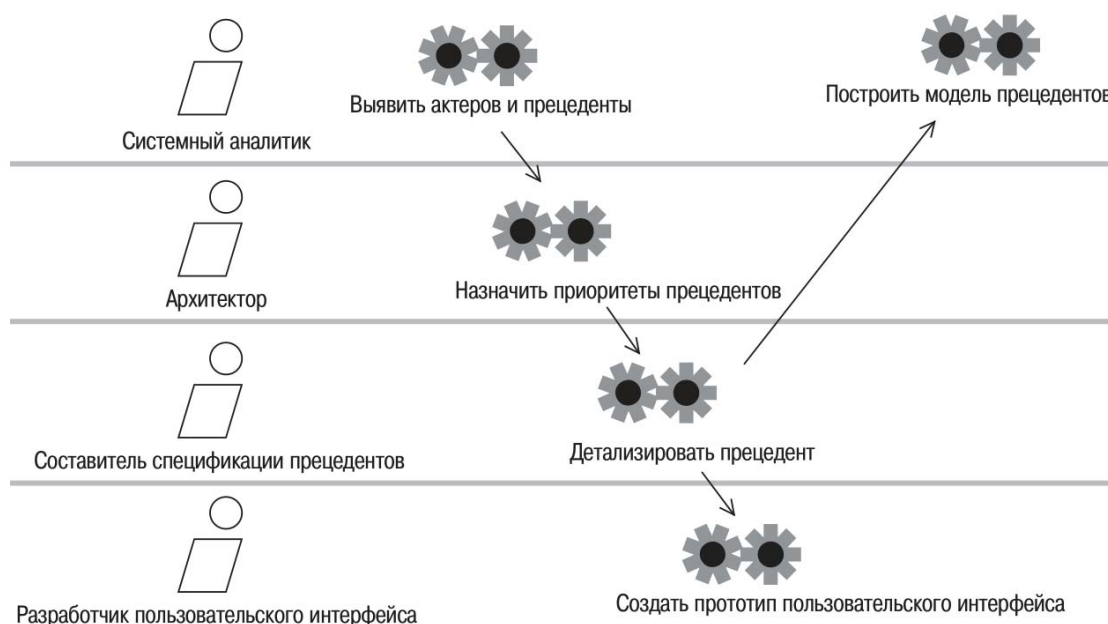


Рис. 2.3. Задачи рабочего потока определения требований

Детализации потока работ UP моделируются в виде исполнителей (пиктограммы в левой части рис. 2.3) и деятельности (пиктограммы в форме шестеренок). Разновидности UP, такие как RUP, могут использовать другие пиктограммы, но с той же семантикой. Стрелки – это отношения, показывающие нормальный поток выполнения работы от одной задачи к следующей. Однако следует помнить, что это только приближенное представление рабочего потока для «усредненного» случая. Оно может и не быть точным представлением происходящего в действительности. В реальности в зависимости от обстоятельств некоторые задачи могут выполняться в другом порядке или параллельно.

Поскольку данное учебное пособие посвящено анализу и проектированию, основное внимание будет сосредоточено только на задачах, важных для ОО-аналитиков и разработчиков. К таким задачам относятся:

- выявление актеров и прецедентов;
- детализация прецедента;
- построение модели прецедентов.

Другие задачи рабочего потока определения требований не так важны для аналитиков и разработчиков. Назначение приоритетов

прецедентов – это деятельность в основном по планированию архитектуры и проекта, а создание прототипа пользовательского интерфейса – это деятельность, касающаяся программирования.

На рис. 2.3 показано, что стандартный рабочий поток UP сосредоточен на прецедентах, исключая все остальные методы выявления требований. В этом нет ничего страшного, но, как было отмечено выше, такой подход не может в достаточной мере реализовать нефункциональные требования к системе. Чтобы досконально рассмотреть все требования, необходимо расширить рабочий поток определения требований в UP и добавить новые задачи:

- выявление функциональных требований;
- выявление нефункциональных требований;
- назначение приоритетов требованиям;
- отображение (trace) требований в прецеденты.

Также был введен новый исполнитель – разработчик требований. Новые задачи и исполнители показаны на рис. 2.4.

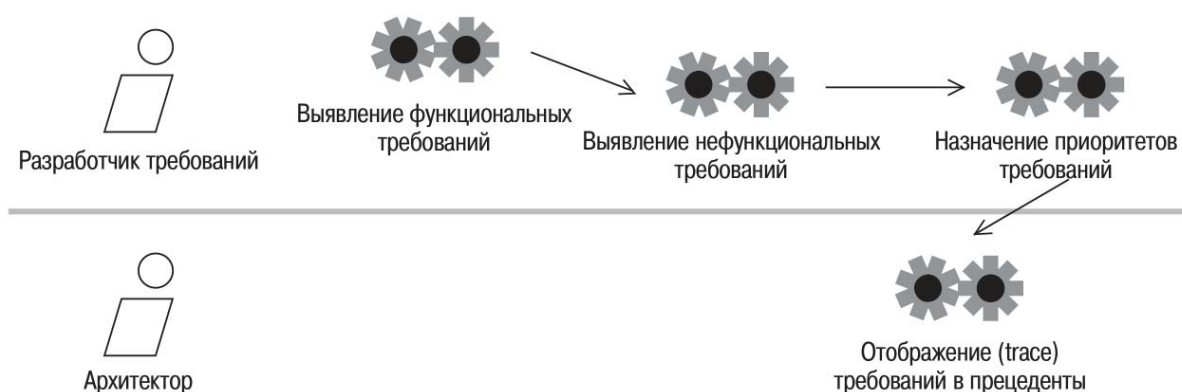


Рис. 2.4. Расширенный рабочий поток определения требований:
новые задачи и исполнители

Выработка требований – это термин, используемый для описания деятельности по выявлению, документированию и обслуживанию ряда требований, предъявляемых к программной системе. Речь идет о выяснении того, что хотят получить от системы заинтересованные стороны.

Неполные требования и неучастие пользователей – это две основные причины провала проектов. Обе проблемы – результат ошибок в выработке требований. Так как конечная программная система основывается на наборе требований, то эффективная выработка требований – это решающий фактор успеха в проектах по разработке программного обеспечения.

Требования – это основа всех систем. Это, по сути, формулировка того, что должна делать система. В принципе, требования должны быть только изложением того, что система должна делать, но не того, как она должна это делать. Можно определить, что должна делать и какое поведение должна демонстрировать система, но при этом не обязательно что-либо говорить о том, как эта функциональность может быть реализована (рис. 2.5).



Рис. 2.5. Формат формулировки требований

Несмотря на тот факт, что поведение системы, и особенно удовлетворение конечного пользователя, закладывается в процессе выработки требований, многие компании по-прежнему не считают это важным. Как уже было упомянуто, основная причина провалов проектов по разработке программного обеспечения заключается в проблемах с требованиями.

UML не дает каких-либо рекомендаций по написанию традиционных требований. Фактически требования в UML представляются исключительно через механизм прецедентов, который будет рассмотрен позже.

Однако многие разработчики моделей полагают, что прецедентов недостаточно и нужны традиционные инструментальные средства выработки и управления требованиями.

2.2. СВОЙСТВА ТРЕБОВАНИЙ

Наука извлечения и формализации качественных (иногда говорят хороших, правильных) требований носит во многом эмпирический характер. Однако в практике разработки программных систем накопились определённые представления о том, какими свойствами должны обладать требования к программной системе. К таким требованиям относятся:

- полнота;
- ясность;

- корректность;
- согласованность;
- верифицируемость;
- необходимость;
- полезность при эксплуатации;
- осуществимость;
- модифицируемость;
- трассируемость;
- упорядоченность по важности и стабильности;
- наличие количественной метрики.

Полнота. Как известно из теории искусственного интеллекта, неполнота – одно из фундаментальных свойств человеческого знания. При создании программных систем нам приходится иметь дело с характеристиками ещё несуществующей системы. Идея о том, что необходимо сформулировать все требования полностью, т. е. исчерпывающим образом, до начала проектирования, а тем более – реализации системы, изжила себя вместе с так называемым каскадным подходом, который поддерживал последовательную модель реализации системы. Спиральный подход, на котором базируется большинство современных методологий, предусматривает поэтапное выделение и детализацию требований на всём протяжении цикла разработки системы.

Тем не менее, требование полноты предъявляется к требованиям, формулируемым к системе. Надо понимать, что данное требование – это скорее тенденция, цель, к которой нужно постараться максимально приблизиться на как можно более ранних стадиях проекта.

Требование полноты можно рассматривать в двух аспектах: полнота отдельного требования и полнота системы требований.

Полнота отдельного требования – свойство, означающее, что текст требования не требует дополнительной детализации, т. е. в нём предусмотрены все необходимые нюансы, особенности и детали данного требования.

Полнота системы требований – свойство, означающее, что совокупность артефактов, описывающих требования, исчерпывающим образом описывает всё то, что требуется от разрабатываемой системы.

Ясность (недвусмысленность, определённость, однозначность спецификаций). Каждый из совладельцев разрабатываемой системы обладает своим личным опытом восприятия событий внешнего мира. Слово, произнесённое вслух, вызывает индивидуальные

ассоциации в семантическом пространстве каждого отдельного воспринимающего субъекта. То, что является ясным, допустим, для кардиохирурга, совсем необязательно будет таковым для специалиста в области программной инженерии. Соответственно, требование обладает свойством ясности, если оно сходным образом воспринимается всеми совладельцами системы. На практике ясность требований достигается в том числе и в процессе консультаций, в ходе которых происходит «выравнивание тезаурусов» совладельцев системы. Хорошим подспорьем в этом служит согласованный сторонами глоссарий ключевых понятий предметной области.

К. Вигерс даёт следующий совет по повышению ясности документов: «Пишите документацию просто, кратко и точно, применяя лексику, понятную пользователям» [1].

Ещё одной стороной понятия «ясность требования» является его *прослеживаемость*. Требование, которое сформулировано ясно, может быть прослежено от того документа, где оно сформулировано впервые, и до рабочих спецификаций.

Корректность и согласованность (непротиворечивость). Корректность – одно из важнейших свойств требований. К. Вигерс вводит понятие корректности требования через точность описания функциональности. В этом смысле корректность в определённой степени конкурирует с полнотой. Но есть и различие. Если свойство полноты носит скорее качественный характер (абсолютная полнота представляет недостижимый идеал, к которому можно приблизиться), то свойство корректности носит оценочный характер и задаёт дихотомию: каждое из требований либо корректно, либо нет. Кроме того, можно рассуждать о взаимной корректности требований или согласованности (непротиворечивости): если два требования вступают в конфликт, значит, как минимум, одно из них некорректно. В иерархии требований можно выделить вертикальную и горизонтальную согласованность. Иными словами, требования не должны противоречить требованиям своего уровня иерархии и требованиям «родительского» уровня соответственно. Так, требования пользователей не должны противоречить бизнес-требованиям, а функциональные требования – требованиям пользователя.

Верифицируемость (пригодность к проверке). Свойства требований нельзя считать независимыми. В математической статистике такие признаки называются коррелирующими. Так, свойство верифицируемости связано со свойствами ясности и полноты: если

требование изложено на языке, понятном и одинаково воспринимаемом участниками процесса создания информационной системы, причём оно является полным, т. е. ни одна из важных для реализации деталей не упущена, значит, это требование можно проверить.

При этом в ходе проверки у сторон (принимающей и сдающей работу) не должно возникнуть неразрешимых противоречий в оценках. Так как хорошо сформулированные требования составляют основу успешного создания системы – роль верифицируемости трудно переоценить. Требования к системе представляют основу контракта между заказчиком и исполнителем, и если данные требования нельзя проверить, значит, и контракт не имеет никакого смысла, следовательно, успех или неудача проекта будут зависеть только от эмоциональных оценок сторон и их способности договориться, а это слишком шаткая основа для осуществления работ.

Необходимость и полезность при эксплуатации. Это одни из самых субъективных и трудно проверяемых свойств требований. Возвращаясь к иерархии требований, наиболее бесспорными следует считать бизнес-требования. Данные требования формулируют первые лица, представляющие заказчика, и вряд ли кто-нибудь лучше них сможет сказать, каким условиям должна соответствовать создаваемая информационная система, чтобы удовлетворять бизнес-целям предприятия. Тем не менее, если у представителя исполнителя возникают сомнения в необходимости того или иного бизнес-требования, вызванные интуитивными соображениями либо опытом внедрения информационных систем на аналогичных предприятиях, он должен проявить инициативу и собрать совместное совещание сторон. Аргументы в пользу отсутствия необходимости требования несомненно будут восприняты, особенно если они будут мотивированы в бизнес-терминологии заказчика и подтверждены выкладками, прогнозирующими соотношение затрат на выполнение требования и ожидаемой от него эффективности.

Необходимость требований пользователя может вытекать из соответствующих бизнес-требований. Кроме того, требования пользователя могут мотивироваться эргономичностью продукта и особенностями функционирования его отдела (подразделения), недостаточно полно раскрытыми на предыдущем уровне иерархии требований. Большинство функциональных требований вытекают из требований первых двух уровней. Другие функциональные требования могут лежать вне сферы компетенции заказчика (который, вообще говоря,

не обязан быть экспертом в области IT), и их должен сформулировать исполнитель. Так, например, информационная система в процессе её использования может начать снижать свою производительность из-за больших объёмов накапливаемых данных. Поэтому целесообразно заложить функции архивирования информации, переключения учётных периодов и т. п., необходимость которых следует не из особенностей бизнеса предприятия внедрения, а из общих принципов построения информационных систем.

Более слабой, чем необходимость, формулировкой обладает свойство *полезности при эксплуатации*. Разграничение между данными свойствами можно провести следующим образом. Необходимыми следует считать свойства, без выполнения которых невозможно либо затруднено выполнение автоматизированных бизнес-функций пользователей; полезными при эксплуатации следует считать любые свойства, повышающие эргономические качества продукта.

Осуществимость (выполнимость). Является в некоторой степени конкурирующим по отношению к введённым выше двум свойствам. В принципе никто не мешает сформулировать требование, выполнимость которого ограничивается сегодняшним уровнем развития техники и технологии, хотя многое из того, что было невыполнимо десять лет назад, вполне выполнимо сегодня.

Выполнимость требования на практике определяется разумным балансом между ценностью (степенью необходимости и полезности) и потребными ресурсами. Если требование является критически важным, необходимым, но выпало из поля зрения при подписании контракта, при этом заказчик готов выделить дополнительно финансирование, а исполнитель – трудовые ресурсы, значит, требование выполнимо. Таким образом, требование осуществимости в ряде случаев также следует считать субъективным, а критерии его оценки лежат в области договорённостей между заказчиком и исполнителем.

Необходимо обеспечить, если понадобится, возможность переработки требований и поддерживать историю изменений для каждого положения. Для этого все они должны быть уникально помечены и обозначены, чтобы можно было ссылаться на них однозначно.

Каждое требование должно быть записано в спецификации только единожды. Иначе легко получить несогласованность, изменив только одно положение из двух одинаковых.

Трассируемость. Трассируемость требования определяется возможностью отследить связь между ним и другими артефактами

информационной системы (документами, моделями, текстами программ и пр.). Отдельная трасса представляет собой направленное бинарное отношение, заданное на множестве артефактов ИС, где первый элемент отношения представляет соответствующее требование, а второй – артефакт, зависимый от данного требования. На практике трассировки анализируются посредством графовых либо табличных моделей. Процесс трассировки позволяет, с одной стороны, выявить уже на стадии проектирования системы проектные артефакты, к которым не ведёт связь ни от одного из артефактов, описывающих требования, с другой стороны, выявить артефакты, описывающие требования, не связанные с проектными артефактами. В первом из случаев целесообразно убедиться в том, что проектный артефакт действительно имеет право на существование, а не является избыточным. Во втором случае необходимо проанализировать полезность выявленных требований: либо эти требования несут недостаточную полезную нагрузку и могут быть игнорированы, либо имеют место ошибки проектирования, т. е. пропущены соответствующие артефакты.

Другая цель трассировки – повысить управляемость проектом: при изменении отдельно взятого требования становится понятным, какие из проектных, рабочих и других артефактов подлежат изменению.

Упорядоченность по важности и стабильности. *Приоритет требования* представляет собой количественную оценку степени значимости (важности) требования. Приоритеты требований обычно назначает представитель заказчика. Разработчик, отталкиваясь от приоритетности требований, управляет процессом реализации информационной системы.

Стабильность требования характеризует прогнозную оценку неизменности требований во времени.

Наличие количественной метрики. Количественные метрики играют важную роль в верификации и аттестации информационных систем. В первую очередь это относится к нефункциональным требованиям, которые, как правило, должны иметь под собой количественную основу (запрос должен обрабатываться не более чем ____ секунд; средняя наработка на отказ должна составлять не менее чем ____ часов). Функциональные требования также могут расширяться количественными мерами при помощи так называемых аспектов применимости.

Полезно разделять требования на функциональные и нефункциональные, определения которых приведены в гл. 1. Существуют

много других способов систематизации требований, но все они используют классификацию, начинающуюся с этих двух категорий.

Функциональное требование – это формулировка того, что должна делать система, это описание назначения системы. Например, для банкомата (Automated Teller Machine – ATM) можно было бы определить следующие функциональные требования:

1. Система ATM shall должна проверять действительность вставленной в банкомат карточки.

2. Система ATM shall должна проверять достоверность PIN-кода, введенного пользователем.

3. Система ATM shall должна выдавать по одной ATM-карточке не более 250 долл. в сутки.

Нефункциональное требование – это ограничение, накладываемое на систему. Система ATM может иметь следующие нефункциональные требования:

1. Система ATM shall должна быть написана на языке C++.

2. Система ATM shall должна обмениваться информацией с банком, используя 256-разрядную кодировку.

3. Система ATM shall должна проверять действительность карточки ATM в течение не более трех секунд.

4. Система ATM shall должна проверять достоверность PIN-кода в течение не более трех секунд.

Можно заметить, что нефункциональные требования определяют или накладывают ограничения на то, как будет реализована система.

При использовании инструментального средства управления требованиями можно организовать требования в *таксономию* – иерархию типов требований, которая может использоваться при классификации требований. Типы требований применяются главным образом для создания небольших групп из громадного числа неструктурированных требований. Такими группами легче управлять, что делает работу с требованиями более эффективной.

Базовое разделение на функциональные и нефункциональные требования, описанное выше, является очень простой таксономией. Но можно пойти дальше и ввести категории требований, расширяя таксономию, как показано на рис. 2.6.

Конкретные типы требований зависят от типа создаваемого программного обеспечения. Это особенно касается функциональных требований. Для нефункциональных требований набор типов требований, приведенный на рис. 2.6, довольно стандартный и достаточный при проектировании большинства систем.

Организация требований по типам полезна в случае большого числа требований (около сотни и более). Такой подход особенно хорош, если используется инструментальное средство выработки требований. Тогда для получения полезной информации можно будет делать запрос к модели требований по типу требований.

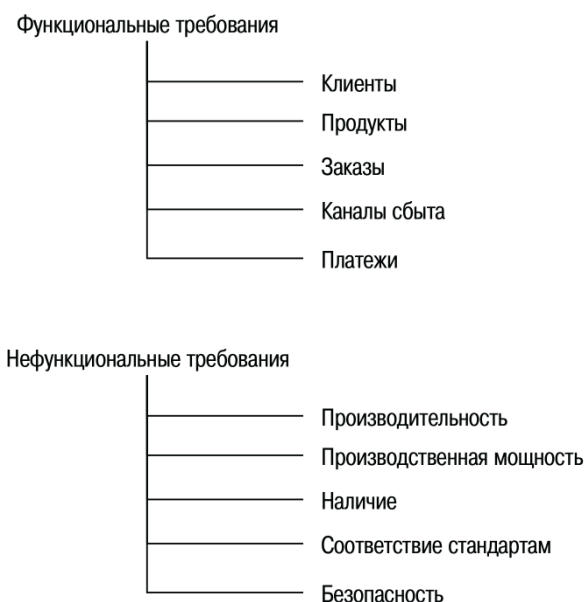


Рис. 2.6. Иерархия типов требований

У каждого требования может быть ряд *атрибутов*, фиксирующих дополнительную информацию (метаданные) о требовании. Каждый атрибут требования имеет описательное имя и значение. Например, у требования может быть атрибут *dueDate* (дата платежа), значением которого является дата выполнения данного требования. Также требование может иметь атрибут *source* (источник), в качестве значения которого выступает описание того, откуда возникло данное требование. Конкретный набор используемых атрибутов зависит от природы и нужд проекта и может меняться в зависимости от типа требования. Основные атрибуты проекта представлены в табл. 2.1.

Наверное, самым распространенным атрибутом требований является *priority* (приоритет). Его значение определяет приоритет требования относительно остальных требований.

Количество атрибутов требований должно быть минимальным, от этого проект только выиграет. Выбор того, что применять — MoSCoW, RUP или другой набор атрибутов требований, зависит от конкретного проекта. Главное при определении набора атрибутов — это сделать его максимально простым. Необходимо выбирать только

те атрибуты, которые полезны проекту. Если атрибут не приносит пользы, его не нужно использовать.

Таблица 2.1

Набор атрибутов требований RUP

Атрибут	Семантика
Status (статус)	Может иметь одно из следующих значений: Proposed (предложенные) – требования, которые находятся в состоянии обсуждения и еще не утверждены; Approved (одобренные) – требования, утвержденные для реализации; Rejected (отклоненные) – требования, которые решено не реализовывать; Incorporated (включенные) – требования, которые были реализованы в определенной версии
Priority (полезность)	Может иметь одно из следующих значений: Critical (критичное) – требование должно быть реализовано, в противном случае система не будет принята заинтересованными сторонами; Important (важное) – требование может быть опущено, но это неблагоприятно отразится на удобстве использования системы и удовлетворении заинтересованных сторон; Useful (полезное) – требование может быть опущено без существенного влияния на приемлемость системы
Effort (трудоемкость)	Оценка времени и ресурсов, необходимых для реализации возможности, выраженная в человеко-часах или другими методами, например методом функциональных точек (www.ifpug.org)
Risk (риск)	Риск, связанный с добавлением этой возможности: High (высокий), Medium (средний) или Low (низкий)
Stability (стабильность)	Оценка вероятности того, что по каким-то причинам требование будет изменено: High (высокая), Medium (средняя) или Low (низкая)
Target Release (целевая версия)	Версия продукта, в которой требование должно быть реализовано

Требования следуют из контекста моделируемой системы. В этот контекст входят:

- непосредственные пользователи системы;
- другие заинтересованные стороны (например, руководители, специалисты обслуживания, установщики);
- другие системы, с которыми взаимодействует данная система;

- аппаратные устройства, с которыми взаимодействует данная система;
- правовые и регулирующие ограничения;
- технические ограничения;
- коммерческие цели.

Как правило, выработка требований начинается с документа, описывающего в общих чертах (видение) то, что собирается делать система и какие услуги она будет предоставлять ряду заинтересованных сторон. Назначение этого документа – обозначить наиболее важные цели системы с точки зрения заинтересованных сторон. Общее описание составляется системным аналитиком в UP-фазе «начало».

После общего описания системы начинается настоящая выработка требований. Основными известными подходами к сбору требований является проведение интервью, анкет и различные стратегии осуществления семинаров при встрече с коллективом заказчика.

Контрольные вопросы и задания

1. На каких фазах разработки наиболее существенно ведется работа с требованиями?
2. Опишите артефакты спецификации требований к программному обеспечению (SRS).
3. Опишите рабочие потоки, связанные с требованиями.
4. Какие требования предъявляются к формулировке описания требований в методологии RUP?
5. Перечислите и кратко опишите основные свойства требований.
6. Дайте понятие трассируемости и опишите используемые способы её отслеживания.
7. Что входит в понятие полноты требования? Приведите соответствующие примеры.
8. Приведите примеры функциональных и нефункциональных требований.
9. Перечислите известные вам атрибуты требований. От чего зависит необходимость использования тех или иных атрибутов при работе над проектом?
10. Опишите процессы сбора требований на начальной стадии проектирования программной системы.

3. МОДЕЛИРОВАНИЕ ПРЕЦЕДЕНТОВ

3.1. ВЫЯВЛЕНИЕ АКТЕРОВ И ПРЕЦЕДЕНТОВ

Моделирование прецедентов – это форма выработки требований. В гл. 2 было показано, как создавать модель требований, объединяя функциональные и нефункциональные требования так называемым традиционным способом. Моделирование прецедентов – это другой, дополнительный способ выявления и документирования требований. Моделирование прецедентов обычно происходит следующим образом:

- 1) устанавливаются границы потенциальной системы;
- 2) выявляются актеры;
- 3) выявляются прецеденты:
 - определяется прецедент;
 - устанавливаются основные альтернативные потоки;
- 4) предыдущие шаги повторяются, пока прецеденты, актеры и границы системы не стабилизируются.

Обычно начинают с самой общей оценки границ системы, чтобы обозначить область моделирования. Затем все действия осуществляются итеративно и зачастую параллельно.

Результатом этой деятельности является модель прецедентов. В этой модели четыре компонента:

1. Граница системы – прямоугольник, очерчивающий прецеденты для обозначения края или границы моделируемой системы. В UML 2 эту границу называют контекстом системы (subject).
2. Актеры – роли, выполняемые людьми или сущностями, использующими систему.
3. Прецеденты – то, что актеры могут делать с системой.
4. Отношения – значимые отношения между актерами и прецедентами.

Модель прецедентов является основным источником для моделирования классов и объектов.

Рассмотрим исходные данные для *выявления актеров и прецедентов* (рис. 3.1):

1. Бизнес-модель – может быть предоставлена в распоряжение разработчиков модели системы, но это не всегда выполняется. Если бизнес-модель есть, то она будет являться превосходным источником для сбора требований.

2. Модель требований – создание этой модели было описано в гл. 2. Эти требования являются хорошим исходным материалом для процесса моделирования прецедентов. В частности, функциональные требования предложат список прецедентов и актеров. Нефункциональные требования – это то, о чем надо помнить при создании модели прецедентов.

3. Список возможностей – это набор потенциальных требований, которые могут быть представлены в форме общего описания проекта.

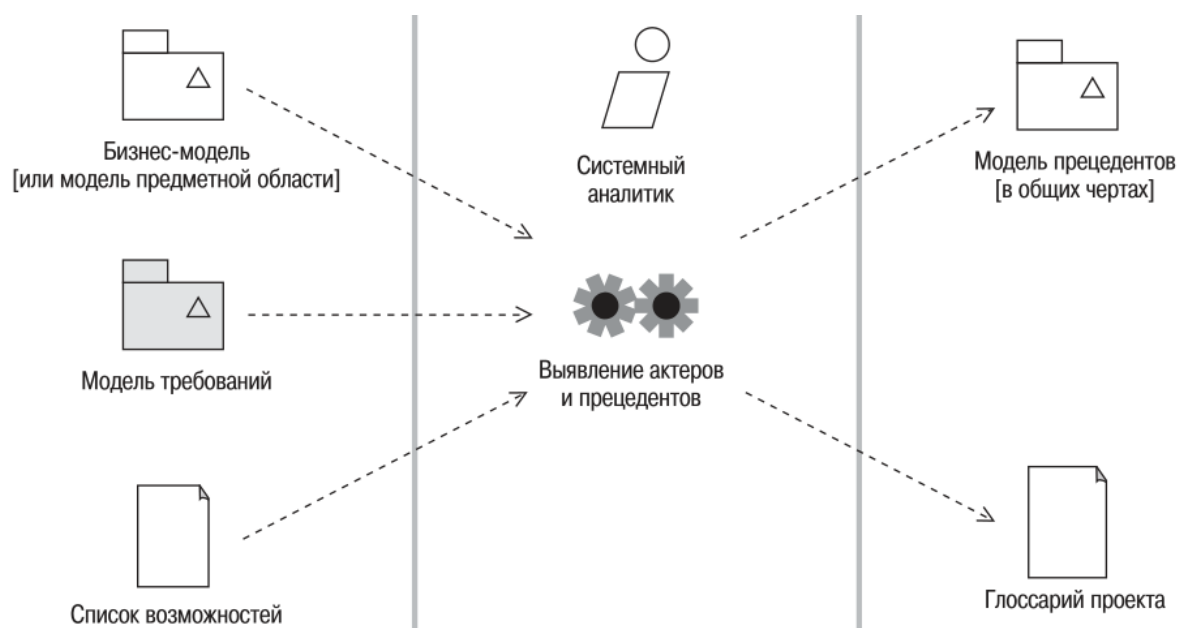


Рис. 3.1. Деятельность UP: выявление актеров и прецедентов

Контекст системы (границы системы). Первое, что необходимо сделать при построении системы, – *обозначить ее границы*. Иначе говоря, надо определить, что является частью системы (находится внутри границ системы) и что находится вне системы (вне ее границ). Это кажется очевидным, но встречается немало проектов, в которых из-за неясности границы системы возникают серьезные проблемы. Точное определение границ системы обычно играет важную роль в выявлении функциональных (а иногда и нефункциональных) требований. В UML 2 границу системы называют *контекстом системы* (subject).

Контекст системы определяется тем, кто или что использует систему (т. е. актерами), и тем, какие конкретные преимущества система предлагает этим актерам (т. е. прецедентами).

Контекст изображается в виде прямоугольника с именем системы. Актеры размещаются вне границ блока, а прецеденты – внутри. В начале моделирования прецедентов имеется лишь предварительное представление о том, где находятся границы системы. По мере выявления актеров и прецедентов контекст системы обретает все более четкие очертания.

Актеры. *Актеры* – это роли, исполняемые какими-либо сущностями, непосредственно взаимодействующими с системой.

Актер определяет роль, которую выполняет некоторая внешняя сущность при непосредственном взаимодействии с данной системой. Он может представлять роль пользователя или роль, исполняемую другой системой или частью аппаратных средств, которые касаются границ системы.

Для понимания сути актеров важно понимать концепцию ролей. Сущности могут играть несколько ролей одновременно либо исполнять их последовательно во времени.

В UML актеры могут быть изображены в виде пиктограммы класса с указанием стереотипа actor или в виде пиктограммы «анимационный человечек». Допускаются обе формы, но многие разработчики моделей предпочитают использовать «человечка» для тех ролей, которые, скорее всего, будут выполняться людьми, а пиктограммы класса – для ролей, которые будут играть другие системы (рис. 3.2).

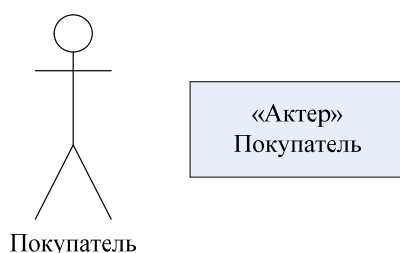


Рис. 3.2. Варианты изображения актера

Важно осознавать, что актеры всегда являются внешними по отношению к системе. Например, покупатель является внешним звеном в системе электронной торговли, такой как книжный интернет-магазин. Однако интересно отметить, что хотя сами актеры всегда находятся вне системы, системы обычно имеют некоторое внутреннее представление одного или более актеров. Например, книжный интернет-магазин сохраняет сведения о большинстве покупателей, содержащие имя, адрес и другую информацию. Это внутреннее

представление внешнего актера Customer. Важно четко разобраться в этом различии. Актер Customer является внешним по отношению к системе, но система может обслуживать класс Customer Details (информация о покупателе), который является внутренним представлением субъектов, играющих роль актера Customer.

Для идентификации актеров необходимо ответить на вопросы, кто или что использует систему и какие роли выполняются актерами при взаимодействии с системой. Чтобы выявить роли, которые люди или сущности играют во взаимодействии с системой, можно учесть, а затем обобщить примеры конкретных людей и сущностей. Следующие вопросы помогут идентифицировать актеров:

- кто или что использует или взаимодействует с системой;
- какие роли они играют во взаимодействии;
- кто устанавливает систему;
- кто или что запускает и выключает систему;
- кто обслуживает систему;
- какие системы взаимодействуют с данной системой;
- кто или что получает и предоставляет информацию системе;
- происходит ли что-нибудь в точно установленное время.

При моделировании актеров необходимо помнить следующие моменты:

1. Актеры всегда являются внешними по отношению к системе, следовательно, находятся вне нашего контроля.

2. Актеры взаимодействуют непосредственно с системой, так они помогают в определении контекста системы.

3. Актеры представляют роли, исполняемые людьми или сущностями по отношению к системе, а не конкретных людей или сущностей. Например, мы составляем и ведем учебные курсы. С точки зрения системы планирования курсов мы играем две роли: Trainer (инструктор) и Course Author (автор курса).

4. У каждого актера должно быть короткое, осмысленное с прикладной точки зрения имя.

5. Каждого актера должно сопровождать краткое описание (одна или две строчки), объясняющее, что данный актер представляет с прикладной точки зрения.

6. Как и в классах, в обозначении актеров могут быть представлены атрибуты актера и события, которые он может получать. Такие обозначения нечасто используются и редко отображаются на диаграммах прецедентов.

Если необходимо смоделировать что-то происходящее с системой в определенный момент времени, но не инициируемое ни одним из актеров, можно ввести актера под названием Time (время). В качестве примера можно привести автоматическое сохранение резервной копии системы, выполняемое ежедневно вечером.

3.2. ОПИСАНИЕ ПРЕЦЕДЕНТОВ

В книге «UML Reference Manual» *прецедент* определен как «описание последовательности действий, включая альтернативные и ошибочные последовательности, которые система, подсистема или класс могут осуществлять, взаимодействуя с внешними актерами» [1]. Прецедент описывает поведение, демонстрируемое системой с целью получения значимого результата для одного или более актеров.

Прецедент – это то, что должна делать система по желанию актера, или *вариант использования* системы конкретным актером:

- прецеденты всегда инициируются актером;
- прецеденты всегда описываются с точки зрения актеров.

Обычно прецеденты рассматриваются на уровне системы, но согласно определению они могут применяться также и для описания варианта использования подсистемы (части системы) или даже отдельного класса. Прецеденты также могут быть очень эффективными при моделировании бизнес-процессов.

Согласно UML прецедент на диаграмме изображается в виде овала. Название прецедента может быть указано внутри или снизу его графического представления.

Идентификацию прецедентов лучше всего начать со списка актеров, а затем рассмотреть, как каждый актер собирается использовать систему. С помощью такой стратегии можно получить список потенциальных прецедентов. Каждому прецеденту должно быть присвоено короткое описательное имя, представляющее собой глагольную группу (какое-либо действие является основной составляющей прецедента).

Во время идентификации прецедентов могут обнаружиться некоторые новые актеры, и это нормально. Иногда приходится очень тщательно анализировать функциональность системы, чтобы выявить всех актеров правильно.

Моделирование прецедентов – итеративный процесс, осуществляемый путем поэтапного уточнения. Все начинается с имени

прецедента, а детали дополняются позже. Эти детали состоят из исходного краткого описания, которое уточняется до полной спецификации.

Диаграмма прецедентов. На диаграмме прецедентов контекст модели прецедентов изображается в виде блока с именем контекста. Этот блок представляет границу системы, моделируемую прецедентами. Актеры располагаются вне контекста (они являются внешними по отношению к системе), а прецеденты, составляющие поведение системы, располагаются внутри контекста (они внутренние по отношению к системе) (рис. 3.3).

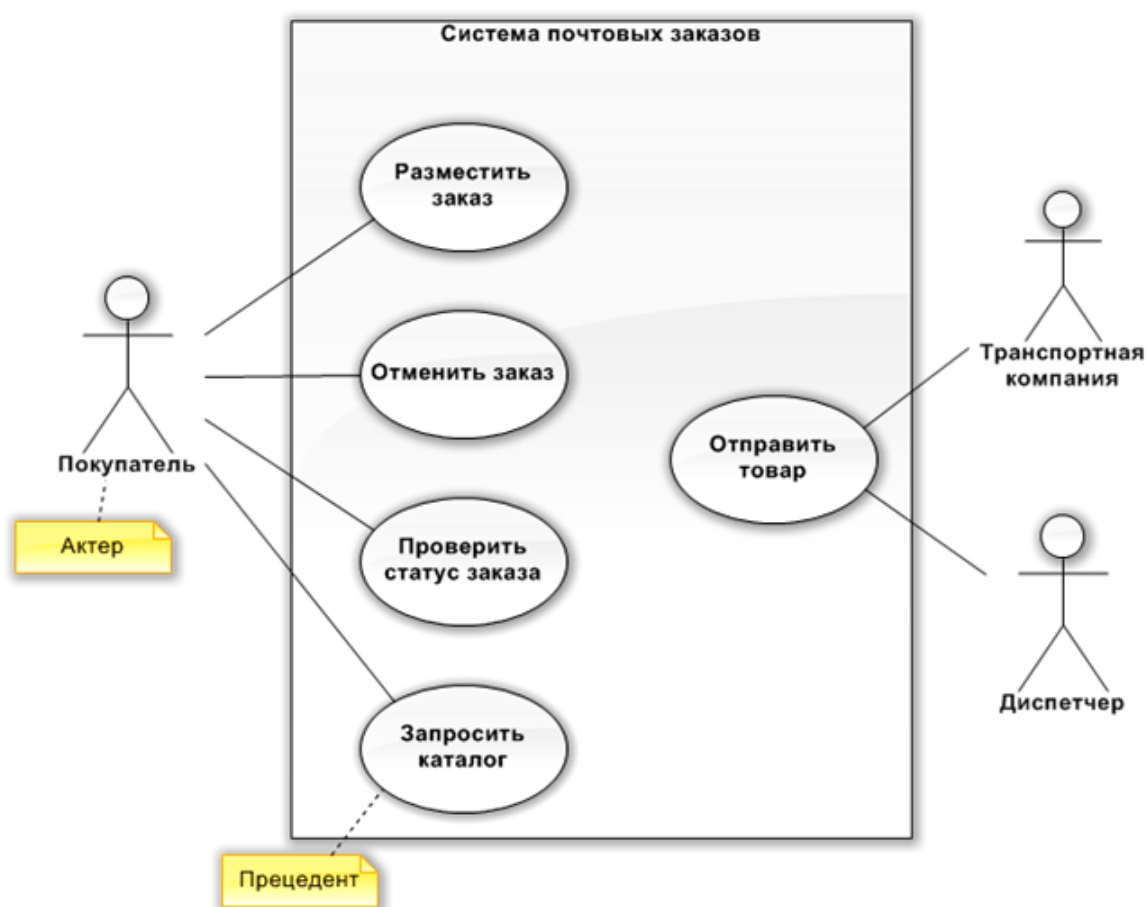


Рис. 3.3. Пример диаграммы прецедентов с указанием обозначений

Отношение между актером и прецедентом обозначается сплошной линией. Это символ *ассоциации* в UML. Ассоциация между актером и прецедентом показывает, что актер и прецедент взаимодействуют друг с другом.

Глоссарий проекта. Глоссарий проекта, возможно, является одним из важнейших артефактов проекта. Любая область деятельности

имеет собственный уникальный язык, и основной целью процесса выработки и анализа требований является понимание и фиксация этого языка. Глоссарий обеспечивает словарь ключевых деловых терминов и определений. Он должен быть понятен всем участникам проекта, включая все заинтересованные стороны, иначе возникнет ситуация, при которой заказчики и разработчики не понимают друг друга, в результате чего итоговый продукт окажется никому не нужным. В случае, если разработчики неправильно поняли требования к проекту, существует риск того, что конечные пользователи не смогут работать с системой.

Кроме определения ключевых терминов глоссарий проекта должен включать синонимы и омонимы.

В глоссарии проекта должен быть указан предпочтительный термин и под его определением перечислены все синонимы. При этом некоторым деловым партнерам придется привыкать к другой терминологии. Обычно тяжело заставить заинтересованные стороны изменить используемый ими язык, но при некоторой настойчивости этого удастся добиться.

UML не устанавливает каких-либо стандартов для глоссария проекта. Лучше всего, если глоссарий будет максимально простым и кратким. Можно применять формат словаря с сортировкой слов и выражений по алфавиту. Возможно, будет достаточно простого текстового документа, но для больших проектов может потребоваться сетевой глоссарий в формате HTML/XML или даже в виде несложной базы данных. Нужно запомнить: чем глоссарий понятнее и проще в использовании, тем сильнее его положительное влияние на проект.

Часть примера глоссария проекта приведена в табл. 3.1. Если у слова нет синонимов или омонимов, это явно указывается словом «нет» (поле не оставляется пустым).

Таблица 3.1

Глоссарий проекта для Clear View Training ECP

Термин	Определение
Каталог	Список всех продуктов, имеющих в продаже в Clear View Training в настоящее время Синонимы: нет Омонимы: нет
Касса	Электронный аналог реальной кассы супермаркета Место, где покупатели могут оплатить продукты, находящиеся в их корзине для покупок Синонимы: нет Омонимы: нет

Термин	Определение
Clear View Training	Компания с ограниченной ответственностью, специализирующаяся на продаже книг и CD Синонимы: CVT Омонимы: нет
Кредитная карта	Карточка, например VISA или Mastercard, которая может использоваться для оплаты Синонимы: карточка Омонимы: нет
Покупатель	Сторона, покупающая продукты или сервисы у Clear View Training Синонимы: нет Омонимы: нет

Термины и определения глоссария проекта также будут использоваться в модели UML. Необходимо гарантировать, что эти два документа синхронизированы. К сожалению, большинство инструментальных средств моделирования UML не обеспечивают какой-либо поддержки синхронизации модели UML и глоссария, поэтому она обычно выполняется вручную.

Спецификация прецедентов. После создания диаграммы прецедентов и выявления актеров и ключевых прецедентов приступают к точному определению каждого прецедента по очереди. Эта деятельность UP известна как детализация прецедента. Некоторые или все прецеденты можно описывать по мере их выявления, не соблюдая строгой очередности.

Итогом этой деятельности является более детализированный прецедент, в описание которого входят, по крайней мере, имя прецедента и его спецификация.

UML-стандарта для спецификации прецедента не существует. Однако широко используется шаблон, приведенный на рис. 3.4. Есть и более сложные шаблоны, однако для первоначального понимания и в проектах малого и среднего размера лучше всего придерживаться максимальной простоты. Важно, чтобы организация выбрала стандарт для спецификаций прецедентов, который будет постоянно использоваться в проектах. Простой и эффективный стандарт для спецификаций прецедентов может помочь обеспечить успешный анализ прецедентов.

В данный шаблон простой спецификации прецедента входит следующая информация:

- имя прецедента;

- ID прецедента;
- краткое описание – абзац, в котором изложена цель прецедента;
- актеры, задействованные в прецеденте;
- предусловия – условия, которые должны выполняться, чтобы прецедент мог осуществиться; это ограничения на состояние системы;
- основной поток – шаги выполнения прецедента;
- постусловия – условия, которые должны выполняться по окончании прецедента;
- альтернативные потоки – список альтернативных основному потоку событий.

Имя прецедента	–	Прецедент: ВыплатаНалогаОборота
Идентификатор прецедента	–	ID: 1
Краткое описание	–	Краткое описание: выплата налога с оборота в Налоговое управление по окончании налогового периода
Актеры, вовлеченные в прецедент	–	Главные актеры: Time (время)
Состояние системы до начала прецедента	–	Второстепенные актеры: TaxAuthority (налоговое управление)
	–	Предусловия: 1. Конец налогового периода
Фактические этапы прецедента	–	Основной поток: (неявный актер Time) 1. Прецедент начинается в <i>конце налогового периода</i> . 2. Система определяет сумму налога с оборота, которую необходимо выплатить налоговому управлению. 3. Система посылает электронный платеж в налоговое управление
Состояние системы после окончания прецедента	–	Постусловия: 1. Налоговое управление получает соответствующую сумму налога с оборота
Альтернативные потоки	–	Альтернативные потоки: нет

Рис. 3.4. Шаблон спецификации прецедента

Прецедент, представленный на рис. 3.4, касается выплаты налога с оборота – формы налога, взимаемого с продаж во многих странах. В данном примере налоговое управление в любом случае тем или

иным образом получает соответствующую сумму налога, поэтому факт получения денег здесь заявлен как постусловие прецедента.

Хорошим способом записи прецедентов является использование «структурированного языка», при этом следует придерживаться единообразного стиля, использовать терминологию, описанную в глоссарии, и избегать двусмысленности.

Не существует UML-стандарта по присваиванию *имен прецедентам*. Рекомендуется всегда записывать имена прецедентов в стиле UpperCamelCase: отдельные слова имени прецедента записываются слитно, каждое слово начинается с заглавной буквы.

Прецеденты описывают поведение системы, поэтому имя прецедента всегда должно быть глаголом или глагольной группой, например PaySalesTax (выплата налога с оборота). Всегда надо стремиться выбрать имя короткое и описательное одновременно. Человек, работающий с моделью прецедентов, должен по одному его имени четко понимать назначение моделируемой бизнес-функции или процесса. В этой и следующих главах можно будет найти множество примеров имен прецедентов. Имя прецедента является его уникальным идентификатором в рамках модели прецедентов.

Хотя имена прецедентов в рамках одной модели прецедентов должны быть уникальными, со временем они могут меняться. Следовательно, надо ввести другой постоянный *идентификатор*, уникально идентифицирующий конкретный прецедент в проекте. Обычно используется просто число.

Кратким описанием должен быть один абзац, в котором изложена цель прецедента. Нужно уловить суть прецедента, прикладное значение, которое он имеет для актеров.

С точки зрения отдельного прецедента существует два типа *актеров*:

- 1) главные актеры – актеры, инициирующие прецедент;
- 2) второстепенные актеры – актеры, взаимодействующие с прецедентом после его инициации.

Второстепенные актеры не инициируют прецедент. Каждый прецедент всегда инициируется одним актером. Однако один и тот же прецедент в разные моменты времени может инициироваться разными актерами. Любой актер, который может инициировать прецедент, является главным актером. Все остальные актеры – второстепенные.

Предусловия и постусловия – это ограничения:

1. Предусловия ограничивают состояние системы, необходимое для запуска прецедента. Они как привратники, которые не дают

актеру инициировать прецедент до тех пор, пока не будут выполнены все их условия.

2. Постусловия ограничивают состояние системы после выполнения прецедента.

Предусловия ограничивают состояние системы, необходимое для запуска прецедента. Постусловия ограничивают состояние системы после выполнения прецедента.

На это можно посмотреть по-другому. Предусловия определяют условия, которые должны быть истинными для того, чтобы прецедент мог быть инициирован. Постусловия определяют, какие условия будут истинными после выполнения прецедента. Предусловия и постусловия помогают спроектировать правильно функционирующую систему.

Предусловия и постусловия должны быть простыми выражениями о состоянии системы, которые затем определяются как истинные или ложные. Их называют логическими условиями.

Если прецедент не имеет предусловий или постусловий, хорошим стилем считается надпись «Нет» в соответствующих разделах спецификации прецедента.

Основной поток. Этапы прецедента представляются в виде потока событий. *Основной поток* описывает «идеальный» ход развития событий в прецеденте. *Альтернативные потоки* могут перехватывать ошибки, ответвления и прерывания основного потока. Основной поток иногда называют основным сценарием (primary scenario), а альтернативные потоки – второстепенными сценариями (secondary scenarios).

Основной поток регистрирует этапы прецедента, отражающие «идеальную» ситуацию, когда все идет, как ожидается и хочется, т. е. не возникает ошибок, отклонений, прерываний или ответвлений.

Отклонения от основного потока можно смоделировать двумя способами, которые будут рассмотрены ниже:

- 1) простые отклонения – создаются ветвления основного;
- 2) сложные отклонения – создаются альтернативные потоки.

Основной поток всегда начинается с действий главного актера, направленных на инициацию прецедента. Удачным способом начала потока можно считать следующую форму записи:

1. Прецедент начинается, когда <актер> <действие>.

Поток событий состоит из последовательности коротких этапов, декларативных, пронумерованных и упорядоченных во времени.

Каждый этап потока прецедента должен быть выражен в следующей форме:

<номер> <кто-либо> <совершает некоторое действие>.

Поток событий прецедента может быть представлен в повествовательной форме, однако это не рекомендуется, поскольку данная форма является слишком неопределенной.

Ниже приведен пример нескольких этапов прецедента PlaceOrder (разместить заказ).

1. Прецедент запускается, когда покупатель выбирает опцию «разместить заказ».

2. Покупатель заполняет в форме свои имя и адрес.

Это правильно сформулированные прецеденты. В обоих случаях мы имеем простое декларативное выражение о том, что некоторая сущность осуществляет некоторое действие. А вот пример неверного описания этапа прецедента:

2. Вводятся данные покупателя.

Использование таких выражений для описания любого этапа прецедентов является неверным. На этом конкретном этапе фактически допущены три важных пропуска:

- кто вводит данные покупателя;
- куда вводятся эти данные;
- что конкретно подразумевается под данными покупателя.

Важно различать и избегать пропусков при написании потоков прецедентов, даже если об этом можно сказать или догадаться исходя из контекста. Прецедент должен быть точным описанием части выполняемых функций системы.

Спецификация UML не определяет способа представления *ветвления потока*. Мы пользуемся идиомой, которая позволяет представить ветвление простым способом без записи отдельного альтернативного потока. Для этого используется ключевое слово «Если» (If).

Стоит отметить, что некоторые разработчики моделей прецедентов выступают против ветвления в прецедентах. Они утверждают, что если есть ответвление, значит, должен быть описан новый альтернативный поток. Тем не менее, простое ветвление потока допустимо, потому что оно сокращает общее число альтернативных потоков и обеспечивает более компактное представление требований.

Как уже указывалось, для представления ответвления потока используется *ключевое слово* «Если» (If). Пример, приведенный на рис. 3.5, показывает хорошо структурированный поток событий с двумя ветвями.

Каждая ветвь начинается с ключевого слова «Если» и простого логического выражения, такого как «Если пользователь вводит новое количество, которое может быть истинным (true) или ложным (false)». Структурированный текст под выражением «Если» – это то, что произойдет, если логическое выражение истинно. С помощью отступов и нумерации можно четко обозначить тело выражения «Если» без использования слов «Конец если» (endif) или другого завершающего выражение синтаксиса.

Прецедент: ManageBasket (УправлениеКорзиной)
ID: 2
Краткое описание: Покупатель меняет количество товаров в корзине.
Главные актеры: Покупатель.
Второстепенные актеры: Нет.
Предусловия: 1. Содержимое корзины для покупок является видимым.
Основной поток: 1. Прецедент начинается, когда Покупатель выбирает товарную позицию в корзине. 2. Если Покупатель выбирает «удалить позицию», 2.1. Система удаляет позицию из корзины, 3. Если покупатель вводит новое количество. 3.1. Система обновляет количество товаров в корзине.
Постусловия: Нет.
Альтернативные потоки: Нет.

Рис. 3.5. Прецедент с двумя ветвлениями

Поскольку события ветвления могут произойти, а могут и не произойти в зависимости от обстоятельств, они не могут генерировать постусловия, которые должны выполняться обязательно. Поэтому ветвление может сократить число постусловий прецедента.

Моделирование альтернативных потоков. У каждого прецедента есть основной поток и может быть множество альтернативных потоков. Они являются альтернативными путями в прецеденте, которые перехватывают ошибки, ответвления и прерывания основного потока.

Ключевым моментом является то, что альтернативные потоки часто не возвращаются в основной поток. Это происходит потому, что они обычно обрабатывают ошибки и исключения основного потока и имеют другие постусловия.

Альтернативные потоки могут быть задокументированы отдельно или добавляться в конце прецедента. В качестве примера на рис. 3.6 представлена модель прецедента с альтернативными потоками.

Прецедент: CreateNewCustomerAccount (СозданиеНовогоАк- каунтаПокупателя)
ID: 3
Краткое описание: Система создает новую учетную запись для Покупателя.
Главные актеры: Покупатель.
Второстепенные актеры: Нет.
Предусловия: Нет.
Основной поток: 1. Прецедент начинается, когда Покупатель выбирает опцию «создать новую учетную запись Покупателя». 2. Пока данные Покупателя недействительны, 2.1. Система просит Покупателя ввести его данные, включая адрес электронной почты, пароль и пароль для подтверждения. 2.2. Система проверяет действительность данных Покупателя. 3. Система создает новые учетные данные для Покупателя.
Постусловия: 1. Создана новая учетная запись для Покупателя.
Альтернативные потоки: InvalidEmailAddress InvalidPassword Cancel

Рис. 3.6. Спецификация прецедента с альтернативными потоками

У этого прецедента три альтернативных потока: InvalidEmailAddress (недействительный адрес электронной почты), InvalidPassword (недействительный пароль) и Cancel (отмена).

Отображение требований. При отображении требований устанавливаются взаимосвязи между моделью требований и моделью прецедентов.

Модель требований и модель прецедентов фактически обеспечивают две «базы данных» функциональных требований. Важно сопоставить эти две модели, чтобы выяснить, нет ли в одной из них чего-то, что не охвачено в другой, и наоборот. Такая постановка вопроса – один из аспектов отображения требований.

Отображение функциональных требований осложняется тем фактом, что между отдельными функциональными требованиями и прецедентами установлены отношения «многие ко многим». Один прецедент будет охватывать множество отдельных функциональных требований, и одно функциональное требование может появляться в нескольких разных прецедентах.

Для отслеживания связи требований с прецедентами полезно создать *матрицу трассируемости требований* (матрицу отображаемости). Это простая таблица с номерами ID отдельных требований, расположенными по вертикали, и именами прецедентов (и (или) номерами ID), расположенными по горизонтали. Во всех ячейках, где прецедент и требование пересекаются, ставится крестик. Обычно матрицы прослеживания требований создаются в виде электронных таблиц (табл. 3.2).

Таблица 3.2

Матрица трассируемости требований

		Прецедент			
		П1	П2	П3	П4
Требование	T1	X	X		
	T2			X	X
	T3		X		
	T4				X
	T5			X	

Матрица трассируемости требований – полезный инструмент для проверки согласованности. Если существует требование, не отображающееся ни в один прецедент, значит, упущен прецедент. И наоборот, если есть прецедент, которому не поставлено в соответствие ни одно требование, понятно, что набор требований неполный, либо следует оценить необходимость и причины возникновения данной функции.

В матрицу трассируемости требований может быть добавлена связь требований или прецедентов с другими артефактами проекта, такими как глоссарий, сценарии тестирования, даже функциональные модули и файлы исходного кода. Всё это позволяет намного эффективнее отслеживать связи артефактов проекта при возникновении изменений в требованиях и повышает итоговое качество проекта.

Применение моделирования прецедентов. Прецеденты фиксируют функциональные требования и поэтому неэффективны для систем, в которых доминируют нефункциональные требования.

Прецеденты являются лучшим выбором для фиксирования требований в следующих случаях:

- когда в системе преобладают функциональные требования;
- в системе много типов пользователей, которым она предоставляет разные функциональные возможности (много актеров);
- когда в системе много интерфейсов (много актеров).

Прецеденты не стоит применять в следующих случаях:

- когда в системе преобладают нефункциональные требования;
- в системе мало пользователей;
- когда в системе мало интерфейсов.

Примерами систем, для которых не годятся прецеденты, являются встроенные (embedded) системы и системы со сложным алгоритмом, но с малым количеством интерфейсов и актеров. Для моделирования таких систем намного лучше воспользоваться более традиционными методами выработки требований. Наилучшими способами отображения требований в таких системах считаются UML-диаграммы классов проекта и диаграммы активности либо блок-схемы алгоритмов.

Контрольные вопросы и задания

1. Опишите процесс моделирования прецедентов в системе.
2. На основании каких артефактов осуществляется выявление актеров и прецедентов?
3. Что такое границы системы?
4. Дайте определение понятия актера.
5. Приведите определение понятия прецедента.
6. Что понимается под глоссарием системы? Для каких целей составляется глоссарий?
7. Опишите шаблон спецификации прецедента.

8. Опишите существующие правила для наименований прецедентов.

9. Что понимается под основными и альтернативными потоками? Опишите особенности моделирования альтернативных потоков.

10. Дайте определение понятия матрицы трассируемости требований.

11. В каких системах моделирование прецедентов не является хорошим инструментом описания требований?

4. СПЕЦИФИЦИРОВАНИЕ ТРЕБОВАНИЙ

4.1. СТРУКТУРИЗАЦИЯ ПРЕЦЕДЕНТОВ. ОТНОШЕНИЯ МЕЖДУ АКТЕРАМИ И ПРЕЦЕДЕНТАМИ

Рассмотрим некоторые дополнительные аспекты моделирования. Между актерами и прецедентами диаграммы вариантов использования могут существовать разного рода отношения, показывающие, что действующие лица и варианты использования взаимодействуют друг с другом. Действующие лица могут взаимодействовать с несколькими прецедентами и между собой, равно как и прецеденты могут быть связаны между собой особого типа отношениями. В основном на диаграммах прецедентов используются такие типы отношений, как ассоциации (association relationship), включения (include relationship), расширения (extend relationship), обобщения (generalization relationship) между актерами и прецедентами, между прецедентами и прецедентами, а также между актерами и актерами:

- ассоциация – это структурное отношение, показывающее, что объект неким образом связан с другим объектом;
- include (включить) – отношение между прецедентами, которое позволяет одному прецеденту включать в себя поведение другого;
- extend (расширить) – отношение между прецедентами, которое позволяет одному прецеденту расширять свое поведение одним или более фрагментом поведения другого;
- обобщение актеров (generalization) – отношение обобщения между обобщенным актером и конкретным актером;
- обобщение прецедентов (generalization) – отношение обобщения между обобщенным прецедентом и специализированным прецедентом.

Обозначения отношений include и extend есть не что иное, как обозначения стереотипов, которые широко используются в UML для создания новых элементов модели путем расширения функциональности базовых элементов. Стереотип (Stereotype) – это механизм, позволяющий категоризировать элементы модели. С помощью стереотипов можно создавать своего рода подтипы типов. Это позволяет UML иметь минимальный набор элементов, которые могут быть дополнены при необходимости для создания связующих базовых элементов в системе.

Важно сохранять максимальную простоту модели, поэтому эти отношения надо применять аккуратно и только там, где они

действительно делают модель прецедентов более понятной. Можно легко увлечься *include* и *extend*, но необходимо избегать этого.

Ассоциация. Отношение ассоциации показывает, что между элементами модели существует семантическая связь (рис. 4.1). Ассоциативное отношение может быть направленным. В этом случае направление связи показывает, кто является инициатором. Если отношение направлено от актера к прецеденту, то это означает, что актер инициирует выполнение прецедента. Так, например, покупатель в системе заказов магазина «Интернет-магазин» инициирует выполнение прецедента «Заказ товаров» (рис. 4.1, б).

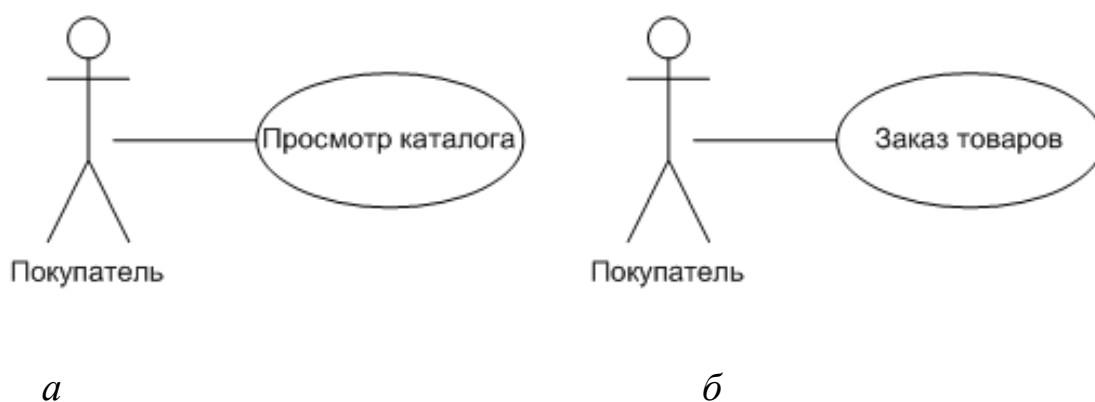


Рис. 4.1. Отношение ассоциации между актером и прецедентом:
а – без указания инициатора действия; б – с указанием инициатора действия

Между прецедентами (вариантами использования) также возможны взаимоотношения, которые описываются отношениями двух типов: включения и расширения (дополнения).

Отношение включения. Отношение включения (*include*) устанавливается только между двумя вариантами использования и указывает на то, что заданное поведение для одного варианта использования включается в качестве составного фрагмента в последовательность поведения другого варианта использования (прецедент явным образом включает в себя поведение целевого). Другими словами, включение создается, когда один прецедент использует другой. При этом исполнение базового прецедента невозможно без исполнения используемого.

Данное отношение является направленным бинарным отношением. Изображается включение в виде пунктирной стрелки с надписью «*include*», которая направлена от базового элемента к используемому. Так, например, в системе заказов магазина «Интернет-магазин»

невозможно осуществление заказа товаров без оплаты. На диаграмме прецедентов это можно отразить так, как показано на рис. 4.2.

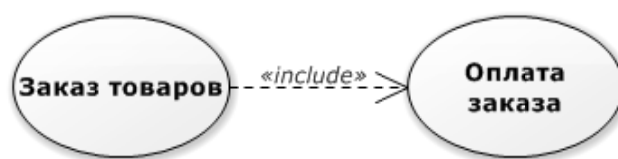


Рис. 4.2. Отношение включения между прецедентами (вариантами использования)

Отношение включения, направленное от варианта использования «Заказ товаров» к варианту использования «Оплата заказа», указывает на то, что каждый экземпляр первого варианта использования всегда включает в себя функциональное поведение или выполнение второго варианта использования. В этом смысле поведение второго варианта использования является частью поведения первого варианта использования на данной диаграмме.

Семантика этого отношения определяется следующим образом. Процесс выполнения базового варианта использования включает в себя собственное подмножество последовательности действий, которая определена для включаемого варианта использования. При этом выполнение включаемой последовательности действий происходит всегда при инициировании базового варианта использования.

Один вариант использования может входить в несколько других вариантов, а также содержать в себе другие варианты. Включаемый вариант использования является независимым от базового варианта в том смысле, что он предоставляет последнему инкапсулированное поведение, детали реализации которого скрыты от последнего и могут быть легко перераспределены между несколькими включаемыми вариантами использования. Более того, базовый вариант зависит только от результатов выполнения включаемого в него варианта использования, а не от структуры включаемых в него вариантов.

Отношение расширения. Отношение расширения (extend) определяет взаимосвязь базового варианта использования с другим вариантом использования, функциональное поведение которого задействуется базовым не всегда, а только при выполнении дополнительных условий. В языке UML отношение расширения является зависимостью, направленной к базовому варианту использования (прецеденту) и соединенной с ним в так называемой точке расширения. Отношение расширения между прецедентами обозначается как

отношение зависимости в форме пунктирной линии со стрелкой, направленной от того прецедента, который является расширением для базового прецедента. Данная линия со стрелкой должна быть помечена стереотипом «extend», как показано на рис. 4.3.

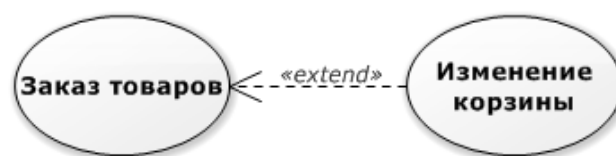


Рис. 4.3. Пример отношения расширения для прецедента «Заказ товаров»

Отношение расширения позволяет моделировать таким образом, что один из вариантов использования должен присоединять к своему поведению дополнительное поведение, определенное для другого варианта использования. В то же время, данное отношение всегда предполагает проверку условия и ссылку на точку расширения в базовом варианте использования. Точка расширения определяет место в базовом варианте использования, в которое должно быть помещено расширение при выполнении соответствующего логического условия. При этом один из вариантов использования может быть расширением для нескольких базовых вариантов, а также иметь в качестве собственных расширений другие варианты. Базовый вариант использования не зависит от своих расширений.

Отношение обобщения. Отношение обобщения служит для указания того факта, что некоторый вариант использования *A* может быть обобщен до варианта использования *B*. В этом случае вариант *A* будет являться специализацией варианта *B*. При этом *B* называется предком или родителем по отношению к *A*, а вариант *A* – потомком по отношению к варианту использования *B*. Следует подчеркнуть, что потомок наследует все свойства и поведение своего родителя, а также может быть дополнен новыми свойствами и особенностями поведения. Графически данное отношение обозначается сплошной линией со стрелкой в форме незакрашенного треугольника, которая указывает на родительский вариант использования.

Отношение обобщения между вариантами использования применяется в том случае, когда необходимо отметить, что дочерние варианты использования обладают всеми атрибутами и особенностями поведения родительских вариантов. При этом дочерние варианты

использования участвуют во всех отношениях родительских вариантов. В свою очередь, дочерние варианты могут наделяться новыми свойствами поведения, которые отсутствуют у родительских вариантов использования, а также уточнять или модифицировать наследуемые от них свойства поведения. Бывают ситуации для данного отношения, когда один прецедент может иметь несколько родительских прецедентов. В этом случае реализуется множественное наследование свойств и поведения. С другой стороны, один вариант использования может быть предком для нескольких дочерних вариантов, что соответствует таксономическому характеру отношения обобщения. Так, в качестве примера может быть рассмотрен прецедент «Оплата заказа» (рис. 4.4). В зависимости от способа оплаты будет несколько изменяться сущность процесса, что выражается прецедентами «Оплата наличными» и «Оплата карточкой».

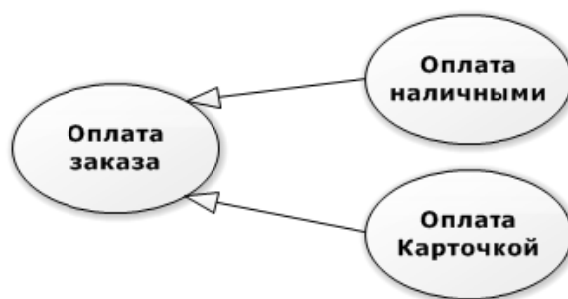


Рис. 4.4. Пример отношения обобщения для прецедента «Оплата заказов»

Между отдельными актерами также может существовать отношение обобщения. Данное отношение является направленным и указывает на факт специализации одних актеров относительно других. Например, отношение обобщения от актера *A* к актеру *B* отмечает тот факт, что каждый экземпляр актера *A* является одновременно экземпляром актера *B* и обладает всеми его свойствами. В этом случае актер *B* является родителем по отношению к актеру *A*, а актер *A*, соответственно, потомком актера *B*. При этом актер *A* обладает способностью играть такое же множество ролей, что и актер *B*.

Зачастую при описании требований к разрабатываемой системе два и более актера могут иметь общие свойства, т. е. взаимодействовать с одним и тем же множеством вариантов использования одинаковым образом. Такая общность свойств и поведения представляется в виде отношения обобщения с другим, возможно, абстрактным актером, который моделирует соответствующую общность ролей.

В качестве примера разберем ситуацию в торговой сфере, где для изменения статуса заказов в магазине с проектируемой системой будут работать менеджер отдела продаж и кладовщик. В этом случае можно создать абстрактного актера и назначить ему прецеденты, которые будут общими для двух актеров, при этом связь обобщения будет выглядеть, как на рис. 4.5.



Рис. 4.5. Пример отношения обобщения для прецедента «Оплата заказов»

Диаграмма прецедентов помогает отобразить основные требования к моделируемой системе и обеспечить взаимопонимание функциональности системы между разработчиком и заказчиком. Можно построить одну, главную, диаграмму прецедентов, на которой будут отражены границы системы (актеры) и ее основная функциональность (прецеденты). Для более подробного представления системы допускается построение вспомогательных диаграмм прецедентов.

Дополнительные обозначения языка UML для бизнес-моделирования. Бывают ситуации, когда при описании требований к разрабатываемой системе необходимо описать бизнес-процессы. В свою очередь, язык UML имеет специальные механизмы расширения, позволяющие ввести дополнительные графические обозначения, нацеленные на решение задач из определенной предметной области (рис. 4.6). Так, для моделирования поведения бизнес-систем на диаграммах прецедентов могут быть использованы следующие элементы: бизнес-актер, сотрудник и бизнес-вариант использования.

Бизнес-актер (business actor) – индивидуум, группа, организация, компания или система, которые взаимодействуют с моделируемой бизнес-системой, но не входят в нее, т. е. не являются частью моделируемой системы. Графическое изображение бизнес-актера приведено на рис. 4.6, а.

Сотрудник (business worker) – индивидиум, который действует внутри моделируемой бизнес-системы, взаимодействует с другими сотрудниками и является участником бизнес-процесса моделируемой системы. Изображение обозначения показано на рис. 4.6, б.

Бизнес-вариант использования (business use case) – вариант использования, определяющий последовательность действий моделируемой системы, направленных на выполнение отдельного бизнес-процесса, его графическое изображение показано на рис. 4.6, в.

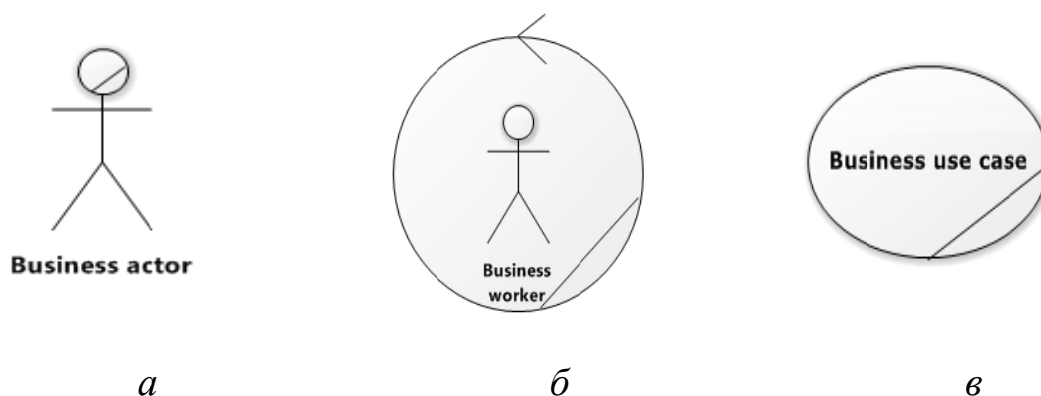


Рис. 4.6. Графические изображения: а – бизнес-актера, б – бизнес-сотрудника; в – бизнес-варианта использования

Примером применения этих элементов графической нотации может служить формирование диаграммы прецедентов для системы продажи товаров в супермаркете. Такая модель может быть использована при создании и автоматизации соответствующих информационных систем.

В данном примере в качестве бизнес-актера для предполагаемой системы можно рассматривать покупателя супермаркета, а в качестве сотрудника – продавца. При этом покупатель является клиентом сервиса «Оформление заказа на покупку товара», а продавец участвует в реализации соответствующего бизнес-процесса. Как следует из выдвигаемых к системе требований, этот сервис выступает в качестве базового бизнес-варианта использования данной системы.

4.2. ДИАГРАММЫ ДЕЯТЕЛЬНОСТИ

В качестве графического представления для выделения основных функций системы применяется диаграмма прецедентов (use case), которая дает представление, что должна делать система.

На вопрос, как она это делает, можно ответить, используя диаграмму активности.

Диаграмма активностей (или, как часто говорят, диаграмма деятельности) – диаграмма UML, выглядящая наиболее простой, поскольку напоминает привычную всем блок-схему. На самом же деле диаграмма активностей – это нечто большее, чем блок-схема, хотя цели у них похожи: обе они отображают некий алгоритм. Диаграмма активностей имеет более широкую нотацию и используется разработчиками для описания:

- бизнес-правил;
- одиночных вариантов использования;
- комплексных серий множественных вариантов использования;
- процессов с узлами решений и альтернативных потоков;
- параллельных операций;
- программных потоков и логических управляющих структур.

Деятельность (Action) может содержать входящие и (или) исходящие дуги деятельности, показывающие потоки управления и потоки данных. Если поток соединяет две деятельности, он является потоком управления. Если поток заканчивается объектом, он является потоком данных.

Деятельность выполняется только тогда, когда готовы все её «входы», после выполнения деятельность передает управление и (или) данные на свои «выходы». Диаграмму активностей (деятельности) принято располагать таким образом, чтобы действия следовали слева направо (рис. 4.7) или сверху вниз.

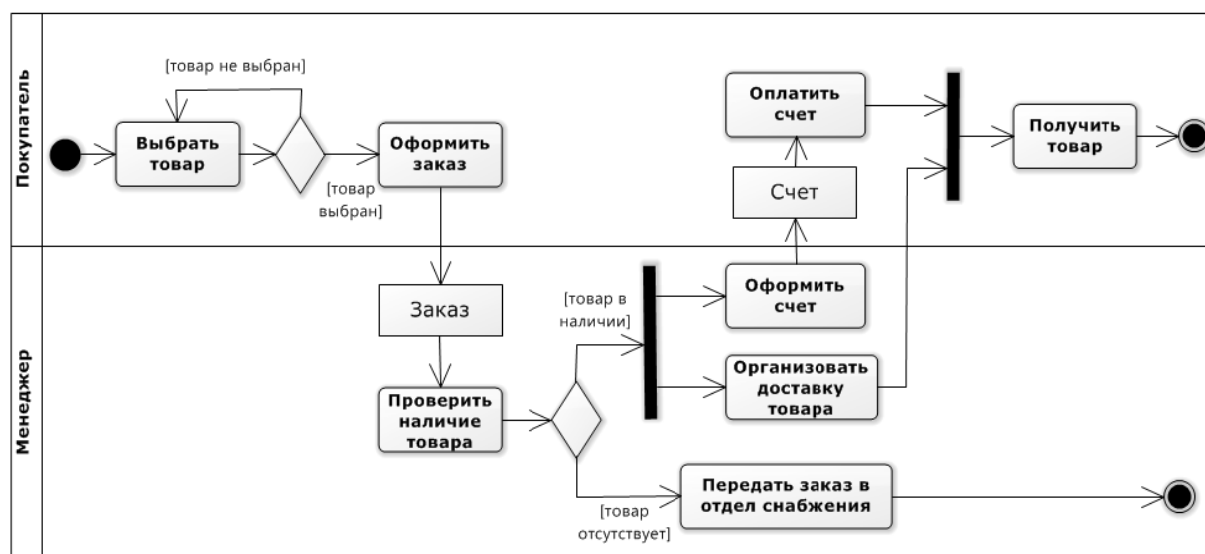


Рис. 4.7. Пример диаграммы активности

Чтобы указать, где именно находится процесс, используется абстрактная точка – *маркер* (или *токен*). Визуально на диаграмме маркер не показывается, данное понятие вводится только для удобства описания динамического процесса. Переход маркера осуществляется между узлами. Маркер может не содержать никакой дополнительной информации (пустой маркер), и тогда он называется маркером управления (*control flow token*), или же может содержать ссылку на объект или структуру данных, и тогда маркер называется маркером данных (*data flow token*).

Для создания диаграммы деятельности используются следующие узлы:

1. Начальный узел деятельности (или начальное состояние деятельности) (*activity initial node*) – является узлом управления, в котором начинается поток (или потоки) при вызове данной деятельности извне.

2. Узел управления (*control node*) – это абстрактный узел действия, которое координирует потоки действий.

3. Конечный узел деятельности (или конечное состояние деятельности) (*activity final node*) – является узлом управления, который останавливает (*stop*) все потоки данной диаграммы деятельности. На диаграмме может использоваться больше одного конечного узла.

4. Конечный узел потока (или конечное состояние потока) (*flow final node*) – является узлом управления, который завершает данный поток. На другие потоки и деятельность данной диаграммы это не влияет.

5. Объект, над которым выполняются действия. Это необязательный элемент диаграммы, но в некоторых случаях необходимо показать объект, инициирующий выполнение действий или являющийся его результатом.

Для отображения расширений сценария на диаграмме деятельности используются так называемые узлы решения. *Узел решения* предназначен для определения правила ветвления и различных вариантов дальнейшего развития сценария. В точку ветвления входит ровно один переход, а выходят два или более. Для каждого исходящего перехода задается булево выражение, которое вычисляется только один раз при входе в точку ветвления. Ни для каких двух исходящих переходов эти сторожевые условия не должны одновременно принимать значение «истина», иначе поток управления окажется неоднозначным. Желательно, чтобы условия покрывали все возможные варианты, иначе поток остановится. Рекомендуется для

пометки исходящего перехода, который должен быть выбран в случае, если условия, заданные для всех остальных переходов, не выполнены, использовать ключевое слово *else*.

Далее следует обратить внимание на такой элемент, как *узел объединения*. Узел объединения имеет не менее двух входящих узлов и один исходящий. Узлы решения-объединения аналогичны логическому выражению «строгое ИЛИ», т. е. для узла объединения только при выполнении того или иного действия осуществляется переход к следующему узлу управления. Соответственно, для узла решения только при выполнении того или иного условия становится доступна возможность перехода к одному из следующих действий.

Для отображения условий, соответствующих логическому оператору «И», на диаграмме используется синхронизационная черта. *Точка разделения* обеспечивает разделение одного потока на несколько параллельных потоков. В нее входит ровно один поток и выходят два и более потока, каждый из которых в дальнейшем выполняется параллельно с другими. Совместно с точкой разделения используется и *точка слияния*. Точка слияния обеспечивает синхронизацию нескольких параллельных потоков. Так, в нее входят два или более потока, причем эти потоки выполняются параллельно, и выходит ровно один поток. При этом в точке слияния входящие параллельные потоки синхронизируются, т. е. каждый из них ждет, пока все остальные достигнут этой точки, после чего выполнение продолжается в рамках одного потока (рис. 4.8).

Также диаграмма активностей может описывать поведение, на которое оказывают влияние внешние события, происходящие за пределами данной системы. На диаграмме это может быть показано при помощи изображения передачи сигнала (семантика была принята в UML 2.0). Передача сигнала может изображаться путем помещения между двумя действиями соответствующего элемента.

Передача сигнала (send signal action) – действие, которое на основе своих входов создает экземпляр сигнала и передает его внешней системе. *Прием события* (receive event action) – действие, которое ожидает некоторого события, принимает и обрабатывает полученное сообщение.

На диаграмме, показанной на рис. 4.9, представлено взаимодействие двух независимых систем: «Учетная система» и «Веб-магазин».

Результатом действия по приему банковской выписки и разнесению оплаты является входящий сигнал для ИС «Веб-магазин», сообщающей об оплате товара.

Соответственно, при получении входящего сигнала в ИС «Веб-магазин» фиксируется факт оплаты, который инициализирует действие «Разрешить отгрузку».



a



б

Рис. 4.8. Примеры диаграмм активностей организации доставки товара:
a – с узлом решения; *б* – с точкой разделения

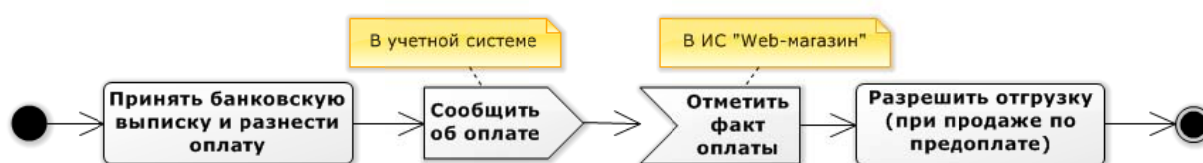


Рис. 4.9. Пример диаграммы активностей с передачей сообщения

Для изображения передачи сигнала можно поместить между двумя узлами деятельности символ деятельности передачи или ожидания сигнала или непосредственно узел объекта, который будет символизировать сигнал.

Разбиение деятельности на разделы. Разделы группируют действия относительно какой-либо общей характеристики, при этом на течение потоков эта группировка никак не влияет. В более ранних версиях UML использовалось такое понятие, как дорожки (swimlanes). Дорожки, используемые при данной структуре

диаграммы активности, зачастую символизируют роль пользователя или организационное подразделение, осуществляющее определенные действия в рамках данной деятельности. Расположение дорожек может быть как вертикальным, так и горизонтальным. Несколько дорожек могут быть объединены по организационному принципу. В UML 2 принято правило применять горизонтальное расположение дорожек для отображения модели бизнес-процесса. Для оптимизации диаграммы деятельности использование дорожек можно заменить указанием наименования раздела перед наименованием действия (рис. 4.10).

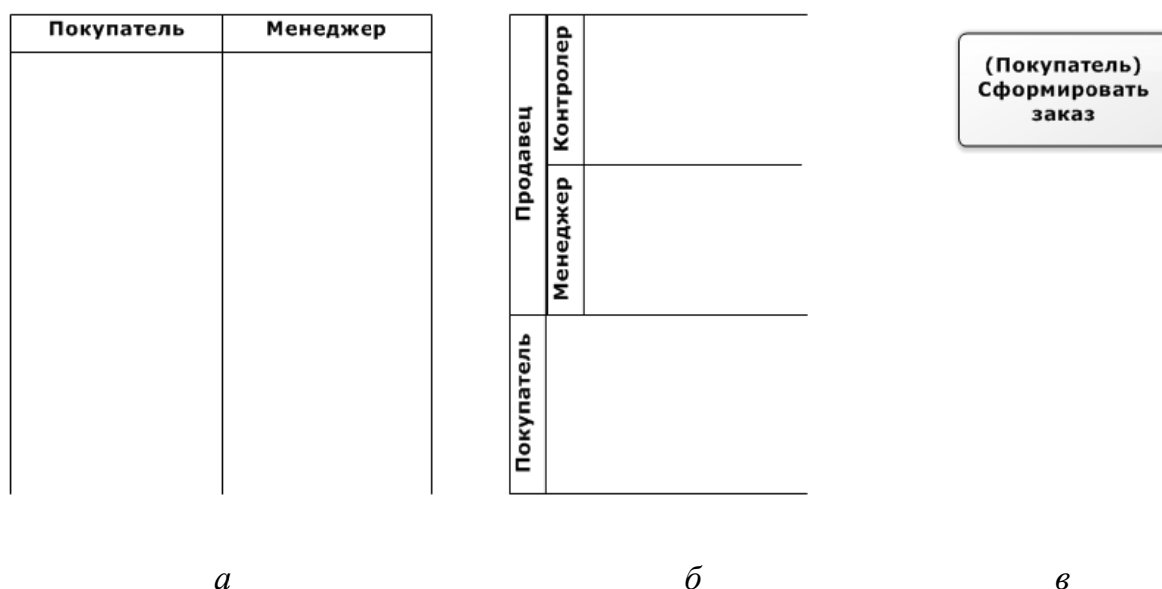


Рис. 4.10. Пример группировки действий для диаграммы активности:
а – вертикальное расположение дорожек; *б* – горизонтальное расположение с вложенными дорожками; *в* – указание имени раздела перед именем действия

Способы детализации поддеятельности. Для описания процессов верхнего уровня на диаграмме обычно отражается переход между деятельностями, которые, в свою очередь, содержат свою последовательность деятельностей или действий. Спецификация UML дает несколько способов представления декомпозиции деятельности на диаграмме. Так, возможно использовать обозначение поддеятельности (subactivity state), где указывается:

- наименование деятельности;
- предусловие и постусловие;
- пиктограмма, информирующая о наличии развернутой диаграммы для данной деятельности.

Такая форма не дает представления о последовательности действий для данной деятельности, а лишь предоставляет ссылку на более детальную диаграмму. При возникновении необходимости описание последовательности действий для поддеятельности может быть размещено непосредственно на основной диаграмме. Для этого описание поддеятельности размещается в отдельном фрейме. При таком способе декомпозиции можно указать:

- предусловия и постусловия;
- входные и выходные параметры (объекты);
- внутреннее устройство деятельности.

Диаграмма активностей является мощным инструментом, который интенсивно используется при создании ИС. В зависимости от поставленной задачи создается диаграмма активности с использованием того набора элементов, который необходим для отражения определенного уровня детализации. Таким образом, диаграмма активностей может применяться как для описания бизнес-процесса, так и для описания функциональных требований к системе.

Место применения. Для создания концептуальной модели необходимо выделять только основные процессы, так как вспомогательные процессы и сущности могут перегружать диаграмму. Также концептуальная модель должна включать только процессы верхнего уровня. Далее можно детализировать каждый процесс на отдельной диаграмме.

Диаграмма деятельности данного вида хорошо отражает:

- последовательность действий;
- события, инициирующие действия или являющиеся конечным результатом;
- условия расширения сценария.

Для того чтобы отобразить соответствие деятельности определенному пользователю или системе, к диаграмме можно применить дорожки. Так как в примере на рис. 4.11 все действия выполняются менеджером, применение разделителей нецелесообразно.

Данный способ иллюстрации бизнес-процесса может охватывать не только действия, происходящие внутри разрабатываемой системы, но и производящиеся за ее пределами, что необходимо для формирования четкого представления о процессе в целом.

Рассмотренный пример содержит только действия, совершаемые в рамках ИС, поэтому данная диаграмма может быть помещена в качестве иллюстрации к сценарию использования в раздел «Общее

описание функций» документа «Техническое задание». Если диаграмма активности будет включать деятельность, выходящую за рамки системы (например, «Оплатить товар»), то она может быть размещена в разделе «Сведения об объекте автоматизации».

Также диаграмма активности целесообразна для описания требований на уровне взаимодействия компонентов системы. Целевой аудиторией в данном случае будет являться команда разработчиков.

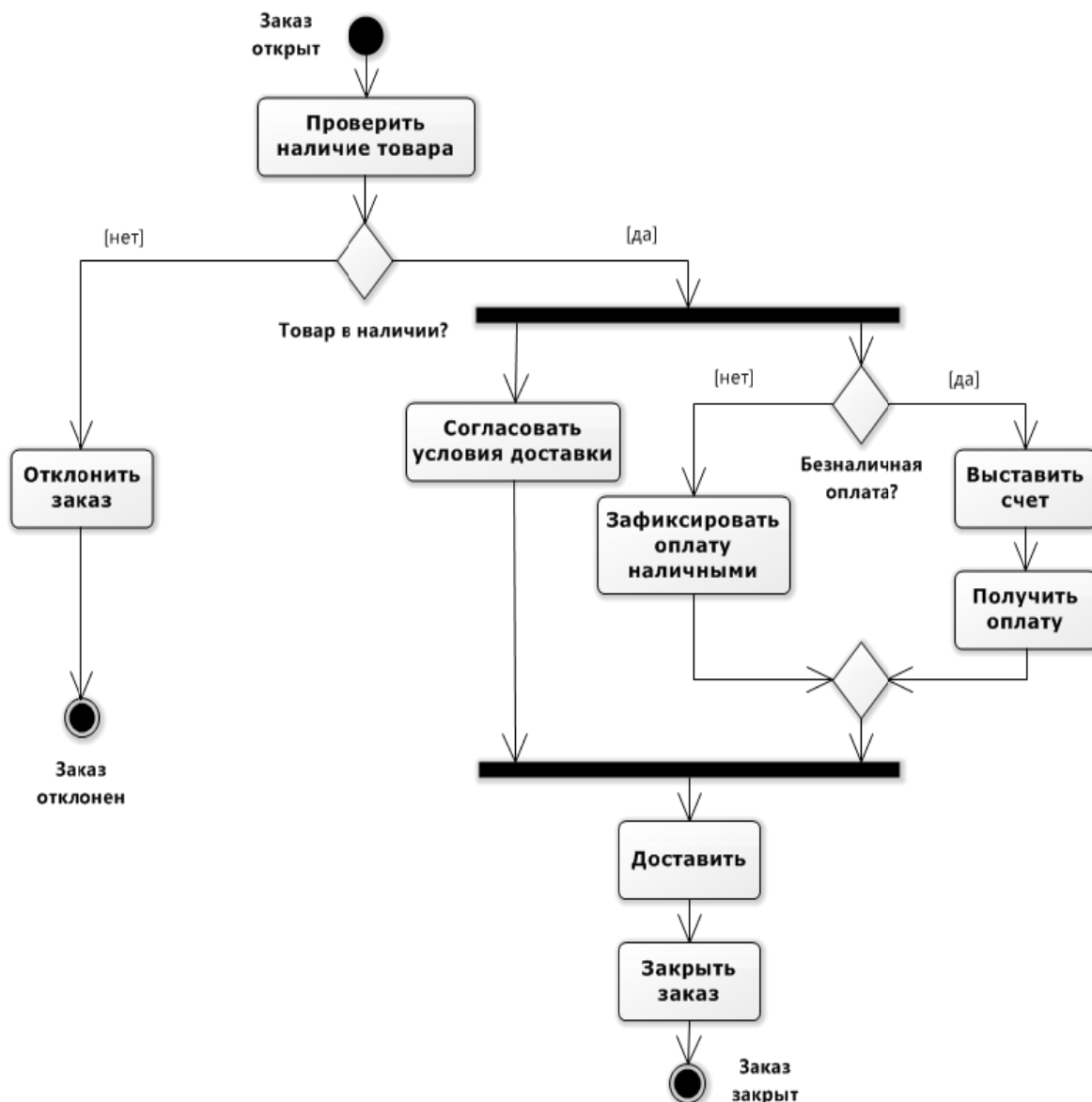


Рис. 4.11. Пример диаграммы активности, описывающей процессы концептуальной модели

Диаграмма активности (деятельности) полезна для описания алгоритма действий, но она не дает представления о поведении

определенного объекта в рамках отдельного варианта использования или системы в целом, что необходимо при объектно-ориентированном программировании.

Также диаграмма активности целесообразна для описания требований на уровне взаимодействия компонентов системы. Целевой аудиторией в данном случае будет являться команда разработчиков.

Диаграмма активности (деятельности) полезна для описания алгоритма действий, но она не дает представления о поведении определенного объекта в рамках отдельного варианта использования или системы в целом, что необходимо при объектно-ориентированном программировании.

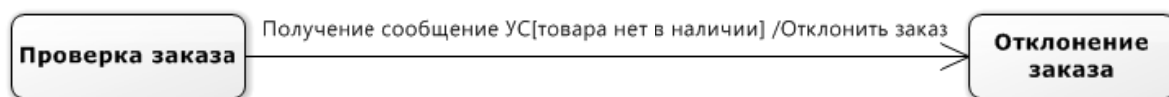
На сегодняшний день при проектировании сложной системы принято делить ее на части, каждую из которых затем рассматривать отдельно. Таким образом, при объектной декомпозиции система разбивается на объекты или компоненты, которые взаимодействуют друг с другом, обмениваясь сообщениями. Сообщения описывают или представляют собой некоторые события. Получение объектом сообщения активизирует его и побуждает выполнять предписанные его программным кодом действия.

При данном подходе система становится событийно управляемой, поэтому разработчикам зачастую важно знать, как должен реагировать тот или иной объект на определенные события. Инициаторами событий могут быть как объекты самой системы, так и её внешнее окружение.

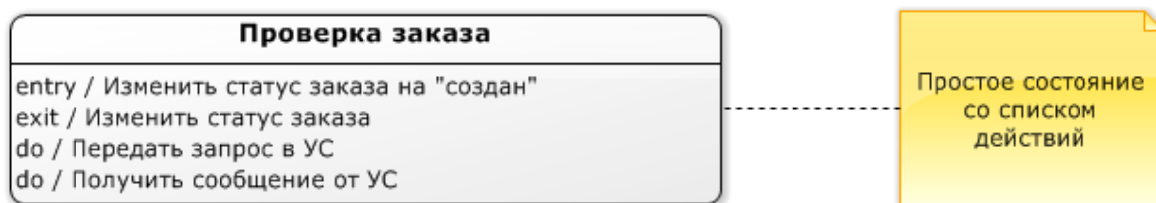
Описать поведение отдельно взятого объекта помогает **диаграмма состояний**. Зачастую диаграмма состояний используется аналитиками для описания последовательности переходов объекта из одного состояния в другое. Диаграмма состояний покажет все возможные состояния, в которых может находиться объект, а также процесс смены состояний в результате внешнего влияния.

Основными элементами диаграммы состояний являются «состояние» и «переход». Диаграмма состояний имеет схожую семантику с диаграммой деятельности, только деятельность здесь заменена состоянием, переходы символизируют действия. Таким образом, если для диаграммы активности (деятельности) отличие между понятиями «деятельность» и «действие» заключается в возможности дальнейшей декомпозиции, то на диаграмме состояний деятельность символизирует состояние, в котором объект

находится продолжительное количество времени, в то время как действие моментально (рис. 4.12).



a



б

Рис. 4.12. Пример диаграммы состояний:

a – связка двух состояний; *б* – детализированное описание состояния

Факт смены одного состояния другим изображается с помощью *перехода*. Переход осуществляется при наступлении некоторого события: окончания выполнения деятельности (do activity), получения объектом сообщения или приема сигнала (подробнее события будут рассмотрены ниже). Переход может быть триггерным и нетриггерным. Если переход срабатывает, когда все операции исходного состояния завершены, он называется *нетриггерным*, или переходом по завершении. Если переход инициируется каким-либо событием, он считается *триггерным* (рис. 4.13).



Рис. 4.13. Пример диаграммы состояний с описанием триггеров переходов

Обязательным параметром является только имя события. В качестве события могут выступать сигналы, вызовы, окончание фиксированных промежутков времени или моменты окончания выполнения определенных действий. После имени события могут следовать круглые скобки для явного задания параметров соответствующего события-триггера (например, пользователь, инициирующий действие). Различаются следующие виды событий:

1. Событие вызова (call event) – событие, возникающее при вызове метода класса. При срабатывании данного вида события объектом асинхронно создается сигнал, который принимается другим объектом. Для указания на то, что некоторая операция посылает сигнал, можно воспользоваться зависимостью со стереотипом send.

2. Событие сигнала (signal event) – событие, возникающее при посылке сигнала. Если событие сигнала представляет возбуждение сигнала, то событие вызова предназначено для описания выполнения операции, т. е. переход осуществляется при получении сигнала от другого объекта. В то время как сигнал является событием асинхронным, событие вызова обычно синхронно. Сигнал также может быть представлен на диаграмме в виде объекта со стереотипом signal.

3. Событие таймера (time event) – возникает, когда истек заданный интервал времени с момента попадания автомата в данное состояние. В UML событие времени моделируется с помощью ключевого слова after (после), за которым следует выражение, вычисляющее некоторый промежуток времени.

4. Событие изменения (change event) – событие, которое возникает, когда некоторое логическое условие становится истинным, будучи до этого ложным. Данное событие моделируется с помощью ключевого слова when.

Если при срабатывании перехода возможно ветвление, в имени перехода используется сторожевое условие. Сторожевое условие (guard condition) всегда записывается в прямых скобках после события-триггера и представляет собой некоторое булево выражение. В общем случае из одного состояния может быть несколько переходов с одним и тем же событием-триггером, при этом целевое состояние будет зависеть от того, какое из сторожевых условий примет значение «истина». Также имя перехода может содержать выражение действия (action expression). В данном случае указанное действие выполняется сразу при срабатывании перехода и до начала каких бы то ни было действий в целевом состоянии. В общем случае выражение действия может

содержать целый список отдельных действий, разделенных символом «;».

С помощью **диаграммы последовательностей** (sequence diagram) можно описывать последовательность, в которой взаимодействуют объекты. Очень важный аспект этого типа диаграммы – время. Так, на диаграмме присутствуют в неявном виде две шкалы. Первая, горизонтальная, отвечает за очередность взаимодействия объектов. Вторая, вертикальная, – это временная ось. Причем началом временного промежутка является верхняя часть диаграммы.

В верхней части располагаются объекты диаграммы, именно они и взаимодействуют. Объект имеет свою линию жизни в виде вертикальной пунктирной линии. Между собой объекты обмениваются сообщениями, которые представлены стрелками. Когда объект активен, он получает фокус управления. Графически такой фокус выражен узким прямоугольником на линии жизни.

Объект – это просто прямоугольник, внутри которого указаны подчеркнутые имя объекта и название класса (необязательно), разделенные двоеточием.

Линия жизни объекта – это линия, которая изображает существование объекта на протяжении некоторого промежутка времени: чем длиннее линия, тем дольше существует объект.

Фокус управления изображается в форме вытянутого узкого прямоугольника, верхняя сторона которого обозначает начало получения фокуса управления объектом (начало активности), а его нижняя сторона – окончание фокуса управления (окончание активности).

В UML каждое взаимодействие описывается совокупностью сообщений, которыми участвующие в нем объекты обмениваются между собой.

Контрольные вопросы и задания

1. Почему нужно строить разные виды диаграмм при моделировании системы?
2. Какие способы изображения актеров вы знаете?
3. Какие виды объектов используются для бизнес-моделирования?
4. В какие отношения могут вступать актеры между собой?
5. В чем состоит смысл отношений включения и расширения?
6. Объясните, для чего может быть использовано отношение обобщения.

7. Перечислите известные вам причины использования прецедентов.

8. Для описания каких особенностей используются диаграммы активностей (деятельности)?

9. Чем диаграммы деятельности отличаются от блок-схем алгоритмов? Какие преимущества это дает разработчикам?

10. Что такое траектория объекта?

11. Чем конечное состояние потока отличается от конечного состояния деятельности?

12. Какая существует особенность использования точек разделения / слияния в диаграмме активностей?

13. Чем моделирование процессов отличается от моделирования операций?

14. В каких ситуациях на диаграмме активностей необходимо отражение передачи сигнала?

15. Для чего и какими способами происходит разбиение деятельности на разделы?

16. В каких разделах документа «Техническое задание» могут быть помещены диаграммы активностей?

17. Применимы ли диаграммы деятельности (активностей) безотносительно к ООП?

18. Для чего необходима диаграмма состояний?

19. Перечислите, какие способы отображения состояния возможны на диаграмме состояний.

20. Назовите известные вам виды событий.

5. МЕТОДОЛОГИИ ПРОЕКТИРОВАНИЯ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

5.1. ПЛАНИРОВАНИЕ ПРОЕКТА НА ОСНОВЕ ТРЕБОВАНИЙ. Методология RUP

Методология проектирования программного обеспечения конкретизирует (в идеале) все аспекты организации разработки, в том числе:

- последовательность работ и их содержание;
- артефакты, которые необходимо создавать в процессе работы: документы, диаграммы, исходные коды и т. д.;
- организацию команды и ролевую ответственность специалистов;
- лучшие практики (best practices), позволяющие максимально эффективно воспользоваться методологией и ее моделью.

Методология разработки вполне конкретна и может быть непосредственно внедрена в организации для производства ПО. За многие десятилетия предложены десятки более или менее известных методологий разработки. Их источниками являлись конкретные авторы, авторские коллективы, коммерческие организации и целые страны в лице своих органов по стандартизации. Остановимся на нескольких наиболее известных и практически значимых для разработчика методологиях.

В настоящее время Rational Unified Process является фактически стандартной методологией разработки ПО. Это объясняется следующими факторами:

- RUP тщательно описывает все рабочие процессы разработки и все создаваемые в этом процессе артефакты, не упуская практически ничего и не заставляя разработчиков что-то додумывать;
- методология использует UML как целостный и универсальный графический язык представления артефактов;
- с самого начала RUP сопровождался мощной средой Rational Rose, которая обеспечила полную инструментальную поддержку методологии, делая ее не только методологией, но и технологией;
- основными авторами RUP и UML стали люди, известные и авторитетные в программной индустрии.

Официально RUP является продуктом фирмы Rational Software Corp., в настоящее время ставшей подразделением IBM. Формально

методология RUP является детализацией более общей методологии Unified Software Development Process (USDP).

В цикле разработки продукта RUP выделяет четыре крупные фазы (показывающие уровень развития продукта), каждая из которых включает в себя одну или несколько итераций.

1. Inception (начало). В данной фазе:

- формируется видение и границы проекта;
- создается экономическое обоснование;
- определяются основные требования, ограничения и ключевая функциональность продукта;
- создается базовая версия модели вариантов использования;
- оцениваются риски.

При завершении фазы оценивается достижение уровня целей жизненного цикла (Lifecycle Objective Milestone), которое предполагает соглашение заинтересованных сторон о продолжении проекта.

2. Elaboration (уточнение, детальная проработка). Производится тщательный анализ предметной области и построение архитектуры. Включает в себя:

- детальное описание большинства вариантов использования (не менее 80 % готовности);
- спроектированную осуществимую архитектуру (executable architecture), реализующую архитектурно-значимые варианты использования;
- обновленное экономическое обоснование и оценки рисков;
- план разработки для всего проекта.

Успешное выполнение фазы означает достижение уровня архитектуры жизненного цикла (Lifecycle Architecture Milestone).

3. Construction (построение, конструирование). Во время этой фазы происходит реализация большей части функциональности продукта. Фаза завершается первым внешним релизом системы и уровнем начальной функциональной готовности (Initial Operational Capability Milestone).

4. Transition (внедрение, передача). Создается финальная версия продукта, которая передается от разработчика к заказчику. Включает в себя программу бета-тестирования, обучение пользователей, а также определение качества продукта. Если качество не соответствует ожиданиям пользователей или критериям, установленным в фазе начала, итерация выполняется снова. Выполнение всех целей означает достижение уровня готового продукта (Product Release Milestone) и завершение полного цикла разработки.

Значительная часть унифицированного процесса связана с созданием и использованием различных моделей разрабатываемой системы, и здесь помогает UML. RUP полностью опирается на UML. В методологии применяются модели вариантов использования, группы моделей анализа, проектирования, тестирования и развертывания.

5.2. ПРИНЦИПЫ БЫСТРОЙ РАЗРАБОТКИ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

Существует группа так называемых agile-методологий (переводимых как *быстрые*, *легкие* или *гибкие* методологии), которые ориентированы на минимальный уровень формализации. Их авторы декларируют следующие принципы:

- взаимодействие людей важнее процессов и инструментов;
- работающая программа важнее исчерпывающей документации;
- сотрудничество с заказчиком важнее обсуждения контрактов;
- адаптация к изменениям важнее следования планам.

Примерами agile-методологий являются Adaptive Software Development (ASD), Lean Development, SCRUM, Kanban, Feature Driven Development (FDD), Dynamic Systems Development Method (DSDM), Crystal Clear. Рассмотрим одну из первых agile-методологий – экстремальное программирование (eXtreme Programming – XP).

Методология XP отвечает следующим принципам:

1. Планирование замысла (planning game). Необходимо как можно быстрее определить объем работ, который нужно сделать до следующей версии. Решение принимается на основе бизнес-приоритетов заказчика и технических оценок. Планы изменяются, как только они начинают расходиться с действительностью или пожеланиями заказчика.

2. Частые выпуски (small releases). Самая первая работающая версия должна появиться как можно быстрее и тут же должна начать использоваться. Следующие версии подготавливаются через достаточно короткие промежутки времени.

3. Метафора системы (system metaphor). Общий вид системы определяется при помощи метафоры или набора метафор, над которыми совместно работают заказчик и программисты.

4. Простой проект (simple design). Правило, предписывающее всегда выбирать самое простое из возможных проектных решений, способное решить текущие задачи.

5. Разработка, управляемая тестированием (test-driven development). Разработчики сначала пишут тесты, потом пытаются реализовать свои модули так, чтобы тесты срабатывали. Заказчики пишут тесты, демонстрирующие возможности системы.

6. Переработка системы (refactoring). Код с целью улучшения постоянно перерабатывается.

7. Парное программирование (pair programming). Программисты работают в парах, непосредственно кодирует один, но обсуждают код вдвоем.

8. Непрерывная интеграция (continuous integration). Система собирается и проходит интеграционное тестирование как можно чаще, как только программисты заканчивают реализацию очередной функции.

9. Коллективное владение кодом (collective code ownership). Каждый программист имеет возможность в любое время усовершенствовать любую часть кода в системе, если он сочтет это необходимым. Он же несет за него ответственность.

10. Заказчик всегда рядом (on-site customer). Представитель заказчика является частью команды и практически все время работы над системой проводит вместе с командой разработчиков.

11. 40-часовая неделя (40-hour weeks). Сверхурочная работа рассматривается как признак больших проблем в проекте. Не допускается сверхурочная работа две недели подряд.

12. Открытое рабочее пространство (open workspace). Команда разработчиков располагается в большом помещении, окруженном комнатами меньшей площади. В центре рабочего пространства устанавливаются компьютеры, на которых работают пары программистов.

13. Стандарт кодирования (coding standard). Программисты должны придерживаться единых соглашений о стиле написания кода.

14. Всего лишь правила (just rules). Команда может в любой момент поменять правила, если ее члены достигли принципиального соглашения по поводу внесенных изменений.

Методология XP сосредоточена на организационных аспектах и на нюансах кодирования и тестирования, а процессы анализа, проектирования и документирования детально не рассматриваются. Это прямое следствие философии agile-методологий, которые пытаются уйти от бюрократизации процесса, обеспечивая «легкость».

Проблема с излишней формализацией и бюрократизацией процесса разработки существует. Долгое время в индустрии бытовало

мнение, что чем больше тщательно оформленной документации выпускается в ходе выполнения проекта, тем лучше. Часто на формальную сторону уходит больше времени и усилий, чем надо, в ущерб главной, «выпускающей» деятельности.

Поскольку внешним проявлением бюрократизации является документирование, то оно в XP принципиально отторгается. Следовательно, отторгается формальный анализ и формальное проектирование, поскольку их результаты иначе как в форме документов (электронных или бумажных) формально зафиксированы быть не могут.

Результат – постоянно меняющиеся требования к продукту и план системы хранятся лишь в «коллективной памяти» разработчиков. Это налагает на квалификацию разработчиков *самые высокие требования*. Данный факт упускают из виду начинающие программисты, которым методологии типа XP кажутся очень привлекательными, поскольку внешне не требуют сложных навыков в анализе, проектировании и документировании. Дилетантам ошибочно кажется, что в XP нужно только и делать, что кодировать, однако успешные проекты, создаваемые в рамках XP (и agile-методологий вообще), возможны лишь при наличии в команде высококвалифицированных профессионалов – аналитиков, проектировщиков и программистов в одном лице, которые ведут весь проект.

Конечно, такие требуются в любой команде, но в XP они незаменимы.

Подводя итоги, отметим следующее:

1. Agile-методологии имеют свою нишу, в которой они могут быть успешными. Совокупность условий, при которых быстрые методологии являются возможным выбором, включает:

- сплоченную команду опытных профессионалов;
- тесный и постоянный контакт с заказчиком или пользователями;
- плохо определенные, постоянно меняющиеся требования;
- относительно небольшой размер и срок проекта.

2. Подобные методологии, как правило, все же не являются полноценными промышленными методологиями. Часто (как в случае с XP) это скорее набор лучших (а иногда и сомнительных) практик, полезных советов, которые все же не составляют единую самодостаточную систему и по отдельности могут с успехом применяться (и применяются) в других методологиях.

Подходы SCRUM. В настоящее время Scrum является одной из наиболее популярных методологий разработки ПО. Scrum – это каркас разработки, с использованием которого разработчики могут

решать появляющиеся проблемы, при этом продуктивно производить продукты большой значимости с точки зрения клиента [2].

Scrum и XP (экстремальное программирование) требуют от команд завершения вполне осязаемого объема работ, который можно предоставить пользователю в конце каждой итерации. Эти итерации планируются таким образом, чтобы быть короткими и фиксированными по времени. Такая целенаправленность на выпуск рабочего кода за короткий промежуток времени означает только одно: в Scrum и XP нет места теории. Эти методологии не гонятся за красивыми UML-моделями, выполненными при помощи специальных CASE-средств, за созданием детализированных спецификаций или написанием кода, который сойдёт на все случаи жизни. Вместо этого Scrum- и XP-команды концентрируются на том, чтобы завершить необходимые задачи. Классический Scrum состоит из элементов, приведенных в табл. 5.1.

Таблица 5.1

Основные элементы Scrum

Роли	Артефакты	Процессы
Владелец продукта Скрам-мастер Команда	Бэклог продукта Бэклог спринта Инкремент продукта	Планирование спринта Обзор спринта Ретроспектива Скрам-митинг Спринт

В Scrum принято выделять три основные *роли*:

1. Владелец продукта (Product Owner) – это человек, ответственный за приоритет требований и часто за их создание.
2. Скрам-мастер – член команды, который дополнительно отвечает за процессы, координацию работы команды и поддержание социальной атмосферы в команде.
3. Команда – семь плюс-минус два человека, которые реализуют требования владельца продукта.

Скрам-мастер должен ежедневно следить за тем, чтобы скрам-митинг начинался и заканчивался вовремя. Рекомендуется выделять определенное время каждому участнику, чтобы общая протяженность митинга не превышала заранее оговоренного времени (например, 15 минут). Скрам-мастер в начале спринта помогает команде проводить планирование спринта и его запуск. В конце спринта скрам-мастер организует демонстрацию результатов спринта при участии

всех заинтересованных лиц и проводит ретроспективу при участии всех членов проектной команды. Также в обязанности скрам-мастера входит мониторинг социальных аспектов команды и поддержание командного духа.

Артефакты:

1. Бэклог продукта (Product Backlog) – приоритизированный список требований с оценкой трудозатрат. Обычно он состоит из бизнес-требований, которые приносят конкретную бизнес-ценность и называются элементами бэклога.

2. Бэклог спринта (Sprint Backlog) – часть бэклога продукта с самой высокой важностью и суммарной оценкой, не превышающей скорость команды, и отобранная для спринта.

3. Инкремент продукта – новая функциональность продукта, созданная во время спринта.

Большинство *процессов* Scrum носят характер встреч, так как данная методология основана на коммуникациях:

1. Скрам-митинг (Scrum Meeting, планерка) – собрание членов команды (с возможностью приглашения владельца продукта) для синхронизации деятельности команды и обозначения проблем. Каждый член команды отвечает на три вопроса:

- 1) что было сделано с предыдущего скрам-митинга;
- 2) какие есть проблемы;
- 3) что будет сделано к следующему скрам-митингу.

Если первый и третий пункт служат для синхронизации деятельности команд, то второй пункт очень важен для выработки решений проблем: если проблема действительно небольшая, ее можно решить или выработать решение прямо на скрам-митинге, если проблема серьезная и требует обсуждения, ее необходимо решить после скрам-митинга.

2. Планирование спринта. Для этого необходимо иметь качественный бэклог, что означает следующее:

- все элементы бэклога должны иметь уникальный номер;
- самые важные элементы бэклога должны быть уточнены и понятны всей команде и владельцу продукта;
- владелец продукта должен четко представлять, что будет реализовано в рамках каждого элемента бэклога.

Основным результатом планирования спринта является бэклог спринта – список задач, которые команда планирует реализовать в рамках спринта. Поскольку длина спринта в Scrum жестко фиксирована,

то команда определяет количество элементов бэклога (объем работ), которые она может реализовать.

В Scrum итерация называется *спринтом* (Sprint), ее длительность составляет 2–4 недели, постоянная продолжительность спринта привносит ритм в разработку. Результатом спринта является готовый продукт (build), который можно передавать заказчику (по крайней мере, система должна быть готова к показу заказчику). Каждый спринт представляет собой небольшой «водопад». В течение спринта делаются все работы по сбору требований, дизайну, кодированию и тестированию продукта. Для оценки объема работ в спринте можно использовать предварительную оценку, измеряемую в очках истории (story point). Оценка спринта должна быть фиксированной, что позволяет команде давать обязательства на тот объем работ, который должен быть сделан в спринте. Это означает, что бэклог не может быть изменен никем, кроме команды. Задачи в течение спринта не меняются.

3. Обзор спринта (демонстрация). Это показ владельцу продукта (заинтересованным лицам) работающего функционала продукта, сделанного за спринт. Основная задача проведения обзора спринта заключается в получении обратной связи. Демонстрация результатов работы не только мотивирует команду, но и подталкивает реализовывать задачи полностью. Если взглянуть на манифест Agile, то там присутствует пункт, который непосредственно касается демонстраций: «Готовый продукт важнее документации по нему» [3]. И действительно, основной мерой прогресса является функционал продукта, поэтому показывать на демонстрации надо именно программу. Если заказчик находится в другом помещении, нужно использовать специальные средства демонстрации. В обзоре спринта обязательно должна принимать участие вся команда, при этом возможны разные стратегии показа. Антипаттерном можно назвать демонстрацию функционала одним человеком, например, скрам-мастером. К паттернам можно отнести демонстрацию «чужих» реализованных элементов бэклога и привлечение к демонстрации аналитиков, тестировщиков, верстальщиков, UI-специалистов и т. д. Такой подход позволяет выработать командную ответственность за результат.

4. Ретроспектива. В долгосрочном плане ретроспективы являются самой важной практикой Scrum, они позволяют адаптировать и кастомизировать Scrum, делая из него по-настоящему гибкий фреймворк для управления проектами. Ретроспективу традиционно

проводят после обзора спринта, спустя небольшое количество времени, чтобы оперативно получить отклик заказчика. Скрам-мастер собирает всю команду для обсуждения результатов спринта. Рекомендуется на ретроспективу приглашать владельца продукта для получения дополнительной обратной связи.

Обычно ретроспектива занимает от 30 минут до 4 часов, и ее продолжительность зависит от следующих факторов:

- длина спринта: чем длиннее спринт, тем больше команда успевает сделать и тем больше материала для обсуждения;
- размер команды: чем команда больше, тем больше надо времени, чтобы у каждого ее члена была возможность высказаться, и тем больше функционала команда успевает сделать;
- наличие проблем: со временем команда решает проблемы, и ретроспективы сокращаются по времени.

Выделим важные особенности Scrum. Если методология Scrum не работает или работает хуже, чем ожидалось, то чаще всего это происходит по одной из следующих причин:

1. Scrum применяется неверно или неполностью. Эмпирический опыт является главным источником достоверной информации. Необходимость полного и точного выполнения Scrum указана в The Scrum Guide [2] и обусловлена нетипичной организацией процесса, отсутствием лидера и руководителя.

2. Недооценена важность работы по обеспечению мотивации команды. Одним из основных принципов Scrum являются самоорганизующиеся, многофункциональные команды. Согласно исследованиям социологов, численность разработчиков, способных на самоорганизацию, не превышает 15 % от работоспособного населения. Таким образом, лишь небольшая часть сотрудников способна эффективно работать в Scrum без существенных изменений в ролях команды, что противоречит идеологии Scrum и потенциально приводит к неверному или неполному использованию Scrum.

3. Scrum применяется для продукта, требования к которому противоречат идеологии Scrum. Scrum относится к семейству Agile, так как приветствует изменения в требованиях в любой момент. Это затрудняет использование Scrum в fixed-cost / fixed-time-проектах. Идеология Scrum утверждает, что заранее невозможно предусмотреть все изменения, таким образом, нет смысла заранее планировать весь проект, нужно ограничиться только планированием работы, которая должна быть выполнена в текущем спринте.

Методология Scrum имеет свои достоинства и недостатки. Scrum ориентирована на клиента и адаптивна, она дает клиенту возможность вносить изменения в требования в любой момент времени (но не гарантирует того, что эти изменения будут выполнены). Возможность изменения требований привлекательна для многих заказчиков ПО.

Scrum достаточно проста в изучении, позволяет экономить время за счет исключения не критичных активностей и получить потенциально рабочий продукт в конце каждого спринта. Данная методология делает упор на самоорганизующуюся, многофункциональную команду, способную решить необходимые задачи с минимальной координацией. Это особенно привлекательно для малых компаний и стартапов, так как избавляет от необходимости в найме или обучении специализированного персонала руководителей.

Конечно, присутствуют и важные недостатки. Ввиду простоты и минималистичности не принято, к примеру, создание плана коммуникаций и реагирования на риски. Таким образом, становится сложным или невозможным формальное (юридическое или административное) противодействие нарушениям правил Scrum.

Другой слабой чертой Scrum является упор на самоорганизующуюся, многофункциональную команду. При кажущемся снижении затрат на координацию команды это приводит к повышению затрат на отбор персонала, его мотивацию, обучение. При определенных условиях рынка труда формирование полноценной, эффективной Scrum-команды может быть невозможным.

Принципы Kanban и их применение на практике. Методология «Канбан» (Kanban) пытается ограничивать ненужную незаконченную работу так, чтобы она никогда не выходила за рамки пропускной способности команды. Слово «канбан» состоит из двух частей: «кан» означает «визуальный, видимый», а «бан» означает карточку или доску.

Методология «Канбан» в agile-разработке приводит к отказу от большого количества практик, которые считаются полезными в Agile. В данной методологии отменяется разработка по фазам с четкими временными границами, пользовательские истории становятся больше, а их число – меньше. Оценка результатов сводится к минимуму или исключается из цикла разработки. Внимание переключается со скорости разработки на продолжительность цикла.

Канбан-разработка строится вокруг *визуальной доски*, которая используется для управления незаконченной работой (рис. 5.1). В Agile пользовательские истории располагают на доске с колонками,

представляющими собой стадии, через которые проходит история, например, «Разработка» или «Тестирование».



Рис. 5.1. Пример канбан-доски

Основной идеей является то, что истории стартуют с левого края доски и проходят всю доску слева направо через те фазы разработки, без которых эти истории нельзя будет назвать завершенными. Завершенные истории, готовые к запуску в эксплуатацию, копятся на правом краю доски. Поскольку «Канбан» ограничивает количество незавершенной работы, ограничивается и количество пользовательских историй, одновременно находящихся на доске.

Канбан-доска обычно включает следующие колонки:

1. Цели (To Do), которые располагаются с левого края доски и на которых фокусируется команда.

2. Очередь историй (Plan), готовых к запуску. Это именно очередь, поскольку истории в ней упорядочены по времени поступления. Первая пришедшая история будет начата раньше всех, при этом она переместится в другие колонки, а остальные истории сместятся вверх по очереди.

3. Стадии разработки истории (Develop, Test, Deploy). Справа от очереди историй расположены колонки со стадиями разработки, через которые должна пройти история, чтобы считаться выполненной. Эти колонки не фиксированы, команда определяет те стадии, через которые проходит история, прежде чем пометаться как выполненная (Done). В некоторых компаниях отдельные колонки могут выделяться под написание документации или под подготовку инженеров поддержки для выпускаемой функциональности.

4. Сделано (Done) – когда история завершена и готова к поставке в эксплуатацию.

В любой работе случаются срочные задачи, для них на доске выделяется специальное место, куда можно поместить одну срочную задачу, и команда должна начать ее выполнять немедленно и завершить как можно быстрее. Такая задача может быть только одна, если появляются еще задачи, то они должны быть добавлены в очередь задач.

Количество задач в каждом столбце должно быть ограничено и зависеть от числа разработчиков в команде. Например, если в команде имеется 8 программистов, то в столбец «Разработка» (Develop) стоит помещать не более четырех задач, что позволит разработчикам не быть слишком загруженными и предоставит время для обмена опытом. Если поставить восемь задач, то каждый программист будет заниматься своей задачей, и некоторые из них будут задерживаться на доске надолго, а ведь главная задача «Канбан» – это уменьшение времени прохождения задачи от начала до стадии готовности.

Уменьшение числа параллельно выполняемых задач сильно уменьшает время выполнения каждой отдельной задачи. Нет нужды переключать контекст между задачами, отслеживать разные сущности, планировать их и т. д. – делается только то, что нужно.

На канбан-доске сразу видны проблемы, возникающие при решении поставленных задач. Например, если тестировщики не справляются с задачами, то столбец «Тестирование» (Test) будет заполнен, и закончившие новую задачу программисты уже не смогут переместить ее в столбец тестирования. Так как речь идет о командной

работе, программисты могут помочь тестировщикам завершить одну из задач тестирования и только тогда передвинуть новую задачу на освободившееся место. Это позволит выполнить обе задачи быстрее.

Методологию «Канбан» можно описать всего тремя основными правилами:

1. Визуализировать разработку. Разделить работу на задачи, каждую задачу написать на карточке и поместить на стену или доску. Использовать названные столбцы, чтобы показать положение задачи при разработке.

2. Ограничивать WIP (Work in Progress или работу, выполняемую одновременно) на каждом этапе разработки ПО.

3. Измерять время цикла (среднее время на выполнение одной задачи) и оптимизировать постоянно процесс, чтобы уменьшить это время.

Контрольные вопросы и задания

1. Расшифруйте аббревиатуру RUP. Какие фазы включает в себя RUP?

2. Дайте характеристику фазе Elaboration.

3. На какой фазе RUP принимается решение о продолжении проекта?

4. Охарактеризуйте методологии Agile, на что они ориентированы?

5. Назовите основные принципы Agile.

6. На чем сосредотачивается методология eXtreme Programming?

7. Опишите принцип test-driven development.

8. Что подразумевает коллективное владение кодом?

9. Каким образом применяется принцип парного программирования в XP?

10. Для каких целей проводятся частые выпуски на этапе разработки ПО?

11. Назовите совокупность условий, при которых стоит использовать agile-методологии.

12. Что влияет на успешность проекта с применением гибких методологий?

13. Дайте характеристику подходу Scrum, опишите основные элементы Scrum.

14. В чем заключается роль скрам-мастера?
15. Что такое бэклог продукта и инкремент продукта?
16. Назовите основные процессы Scrum.
17. Что собой представляет спринт?
18. Для чего применяется ретроспектива?
19. По каким причинам методология Scrum может не сработать?
20. Назовите основные достоинства и недостатки Scrum.
21. На что направлена методология Kanban? Каким образом решаются множественные неотложные задачи в этой методологии?
22. Расшифруйте аббревиатуру WIP. Как данное понятие соотносится с методологией разработки ПО?
23. Опишите основные правила Kanban.

ПОСЛЕСЛОВИЕ

Для успешного управления процессом разработки программного обеспечения зачастую недостаточно выбрать ту или иную методологию и следовать ей на протяжении всего процесса. В случае больших проектов, разработка которых ведется долгий период времени, важным фактором может оказаться способность быстро менять используемую в данный момент методологию. Именно поэтому помимо ясного понимания особенностей той или иной методологии необходимо знание различий между разными подходами и умение быстро переключаться между ними. Часто, вместо того, чтобы искать слабые места в применяемой методологии разработки ПО и пытаться их устранить, можно выбрать другой, более подходящий к изменившимся условиям подход.

Кроме того, любая методология должна быть опробована командой разработки проекта. В процессе работы могут возникнуть особенности при взаимодействии между участниками процесса, которые позволяют лучше использовать одни процессы и затрудняют применение классических методологий.

Правильная постановка требований является важнейшим этапом разработки программного обеспечения, необходимым для успешной реализации процесса разработки. Ошибки, допущенные в постановке и спецификации требований, являются наиболее критичными и приводят к большим затратам по их исправлению на дальнейших этапах разработки программных продуктов. Умение осуществлять специфицирование требований является одним из ключевых навыков инженера программного обеспечения и позволяет правильно ответить на вопрос, предшествующий собственно разработке программного обеспечения: «Что именно требуется построить?».

В учебном пособии сформулированы основные принципы разработки и документирования требований к сложным программным системам в виде различных артефактов, показаны варианты записи требований в виде прецедентов (вариантов использования системы), представлены различные модели UML, которые используются на всех стадиях разработки программного обеспечения.

Читателям предлагается ознакомиться с современными методологиями разработки сложных программных систем: как с наиболее известной из «тяжеловесных» подходов RUP, так и с методологиями быстрой или гибкой разработки – Scrum и Kanban. Следование данным методологиям позволит перенять опыт разработчиков программных систем и использовать наилучшие из известных подходов в своих задачах.

БИБЛИОГРАФИЧЕСКИЕ ССЫЛКИ

1. Вигерс К., Битти Дж. Разработка требований к программному обеспечению. СПб. : БХВ-Петербург, 2016. 736 с.
2. Schwaber K., Sutherland J. The Scrum Guide. The definitive Guide to Scrum: The Rules of the Game [Электронный ресурс]. URL: <https://www.scrumguides.org/docs/scrumguide/v1/scrum-guide-us.pdf> (дата обращения: 05.11.2017).
3. Манифест Agile [Электронный ресурс]. URL: <http://agilemanifesto.org/iso/ru/principles.html> (дата обращения: 05.11.2017).

БИБЛИОГРАФИЧЕСКИЙ СПИСОК

1. Арлоу, Д. UML 2 и унифицированный процесс. Практический объектно-ориентированный анализ и проектирование : пер. с англ. / Д. Арлоу, И. Нейштадт. – 2-е изд. – СПб. : Символ Плюс, 2007. – 624 с. : ил.
2. Бек, К. Экстремальное программирование: разработка через тестирование / К. Бек. – СПб. : Питер, 2017. – 224 с.
3. Ларман, К. Применение UML 2.0 и шаблонов проектирования. Практическое руководство : пер. с англ. / К. Ларман. – 3-е изд. – М. : Вильямс, 2013. – 736 с. : ил.
4. Мирошниченко, Е. А. Технологии программирования : учеб. пособие / Е. А. Мирошниченко. – 2-е изд., испр. и доп. – Томск : Изд-во Томского политехн. ун-та, 2008. – 124 с.
5. Шмуллер, Д. Освой самостоятельно UML 2 за 24 часа. Практическое руководство / Д. Шмуллер. – М. : Вильямс, 2005. – 416 с.
6. Химонин, Ю. И. Сбор и анализ требований к программному продукту (Версия 1.03) / Ю. И. Химонин. – 2009. – 51 с.
7. Якобсон, А. Унифицированный процесс разработки программного обеспечения / А. Якобсон, Г. Буч, Дж. Рамбо. – СПб. : Питер, 2002. – 496 с. : ил.
8. Ladas, C. Essays on Kanban Systems for Lean Software Development / C. S. Ladas. – Modus Cooperandi Press, 2009. – 178 p.

ПРИЛОЖЕНИЕ

ОБЩАЯ СТРУКТУРА ТРЕБОВАНИЙ

Структура требований при использовании средств, ориентированных на документ «Маркетинговые требования» (Product Marketing Requirements, также PVD – Product Vision Document), – отдельный документ:

1. Бизнес-требования (Business Requirements):
 - стимулы (Product Stimuli);
 - целевой рынок (Target Marketing Segment) – для MRD;
 - сценарии использования и проблемы (Scenarios and Problems);
 - обзор конкурентов (Competitors Review) – для MRD, как правило, оформлен как отдельный документ.
2. Решение (Solution):
 - образ продукта (Product Vision);
 - список возможностей (Features List).
3. Суть проекта (Project Scopes) – мероприятия по созданию следующей версии продукта):
 - возможности продукта (Product Features);
 - вехи продукта (Product Milestones).

Требования к продукту (Product Requirements Document) – множество документов, каждый из которых описывает необходимую функциональность:

- профили пользователей (User Profiles);
- пользовательские истории (User Stories);
- варианты использования (Use Cases) – систематизированные пользовательские истории;
- пользовательские требования (User Requirements) – разделяются на функциональные и нефункциональные.

Функциональная спецификация (Functional Specification) – множество документов, каждый из которых описывает необходимую функциональность:

- 1) системные требования (System Requirements);
- 2) требования проектирования (Design Requirements) – архитектура продукта (Product Architecture);
- 3) ограничения интерфейсов (Interfaces Scopes) – часто является расширением требований к проектированию;
- 4) модель взаимодействия (Interaction Model):
 - варианты использования (Use Case Diagram);
 - активность (Activities);
 - макеты графического пользовательского интерфейса (GUI mock-ups);
 - спецификация интерфейса командной строки (CLI specification);
 - спецификация интерфейса прикладного программирования (API specification).
- 5) техническая информация (Technical Notes).

Учебно-теоретическое издание

Буряченко Владимир Викторович

Зотин Александр Геннадьевич

Пахирка Андрей Иванович

**РАЗРАБОТКА ТРЕБОВАНИЙ
ПРИ ПРОЕКТИРОВАНИИ
ПРОГРАММНЫХ СИСТЕМ**

Учебное пособие

Редактор *О. А. Плехова*
Оригинал-макет и верстка *С. Ю. Храброй*

Подписано в печать 29.04.2019. Формат 60×84/16. Бумага офисная.
Печать плоская. Усл. печ. л. 8,2. Уч.-изд. л. 5,8. Тираж 50 экз.
Заказ 201 /19.

Санитарно-эпидемиологическое заключение
№ 24.49.04.953.П.000032.01.03 от 29.01.2003 г.

Редакционно-издательский отдел СибГУ им. М. Ф. Решетнева.
Отпечатано в редакционно-издательском центре
СибГУ им. М. Ф. Решетнева.
660037, г. Красноярск, просп. им. газ. «Красноярский рабочий», 31.